CSUN | COLLEGE OF ENGINEERING AND COMPUTER SCIENCE

ECE 520/L

System-on-Chip Design Lab

Lab 4
Custom AXI-Lite IP

Author: Aaron Nanas
Fall 2026

Table of Contents

1. Introduction 2
2. Objectives 2
3. Required Components 2
4. Procedure 3
4.1. 16-Bit Multiplier Implementation 3
5. Tasks 51
5.1. Hardware Tasks 51
5.2. Software Tasks 54
6. Custom AXI-Lite FIFO IP 56
6.1. Design Specification 57
6.2. Register Map 58
6.2.1. FPGA Revision Register [FPGA_REV_REG] 59
6.2.2. FIFO Reset Register [FIFO_RST_REG] 59
6.2.3. FIFO Write Data Register [WR_DATA_REG] 59
6.2.4. FIFO Read Request Register [RD_REQ_REG] 60
6.2.5. FIFO Read Data Register [RD_DATA_REG] 60
6.2.6. FIFO Status Register [STATUS_REG] 61
6.2.7. FIFO Data Count Register [DATA_COUNT_REG] 61
6.2.8. FIFO Configuration Register [CONFIG_REG] 62
6.2.9. FIFO Threshold Register [THRESHOLD_REG] 62
7. References 63

1. Introduction

The primary goal of this lab is to design, integrate, and verify a custom AXI4-Lite peripheral in a Zynq SoC system. In this lab, you will create a custom 16-bit multiplier AXI IP in the Programmable Logic (PL). Next, you will connect the IP to the Zynq Processing System (PS) using an AXI interconnect. You will then access the registers of the AXI IP using software running on the ARM processor. In the second part of the lab, you will wrap your BRAM FIFO design from the previous lab in a second custom AXI4-Lite IP and control it from software through a set of memory-mapped registers.

2. Objectives

In this lab, you will learn how to:

1. Create a custom AXI4-Lite peripheral in Vivado and modify the generated IP template to implement a 16-bit multiplier
2. Define and manage memory-mapped registers for the operands and the product of the multiplier
3. Package and add custom IP to a Vivado Block Design and connect the custom IP to the Zynq PS using an AXI interconnect
4. Assign and verify memory addresses in the Address Editor
5. Generate the hardware files, export the hardware design to Vitis IDE, and develop a C program to write to and read from the custom IP's registers
6. Integrate an existing RTL design into a custom AXI4-Lite IP and define a register map with read-only, write-only, and read / write registers.

3. Required Components

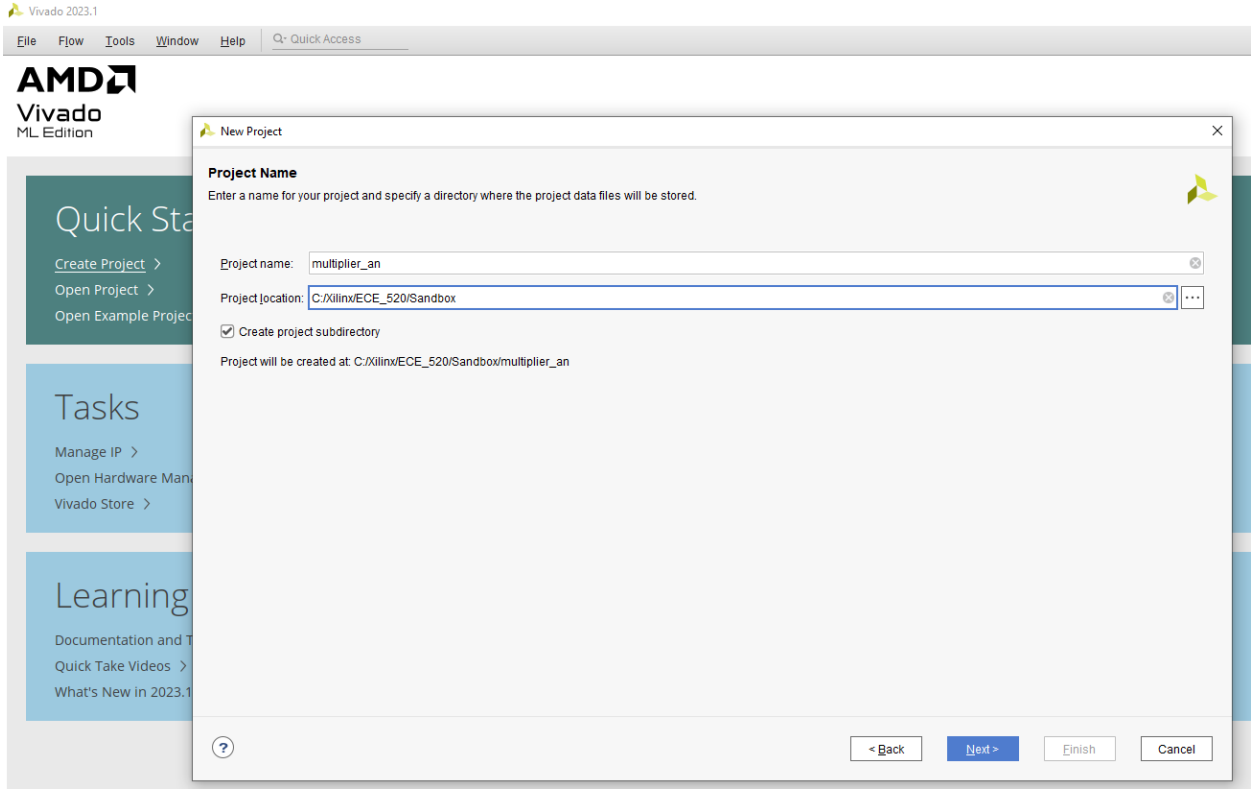
The hardware used for this lab is listed below. Note that the preferred operating system to use for this course is Windows.

Description	Quantity	Manufacturer
Zybo Z7 ARM/FPGA SoC Board	1	Digilent
USB-A to Micro-USB Cable	1	N/A

4. Procedure

4.1. 16-Bit Multiplier Implementation

1. Launch Vivado and create a project called **multiplier_<your_initials>**. An example is shown below. Hit the **Next** button after you've entered the project name and the project location. Configure the Vivado project settings.



2. Implement a 16-bit multiplier module named **multiplier_16_bit** in Verilog. The module should receive two 16-bit unsigned inputs and a registered 32-bit unsigned output. In addition, it should have two more inputs: clk (125 MHz) and rst_n (active low).

   The module declaration is given below for your reference.

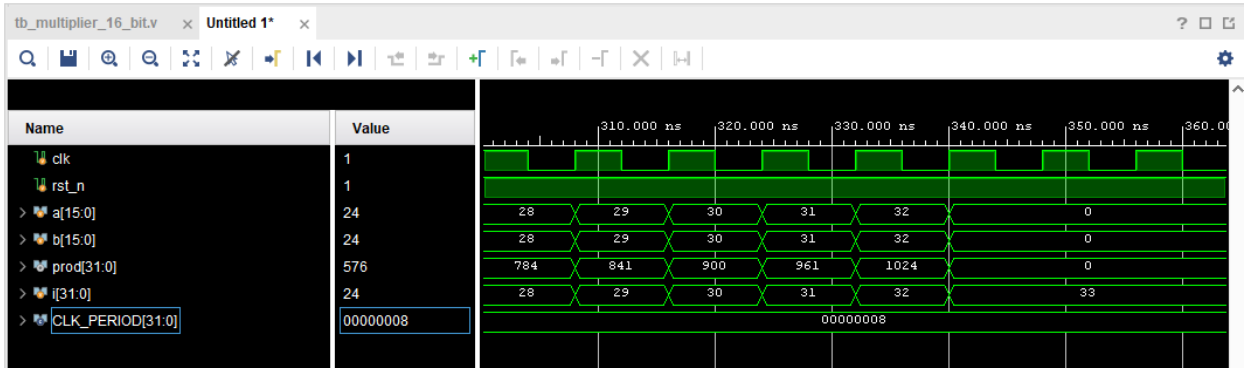
multiplier_16_bit.v

multiplier_an > multiplier_an.srcs > sources_1 > new > multiplier_16_bit.v

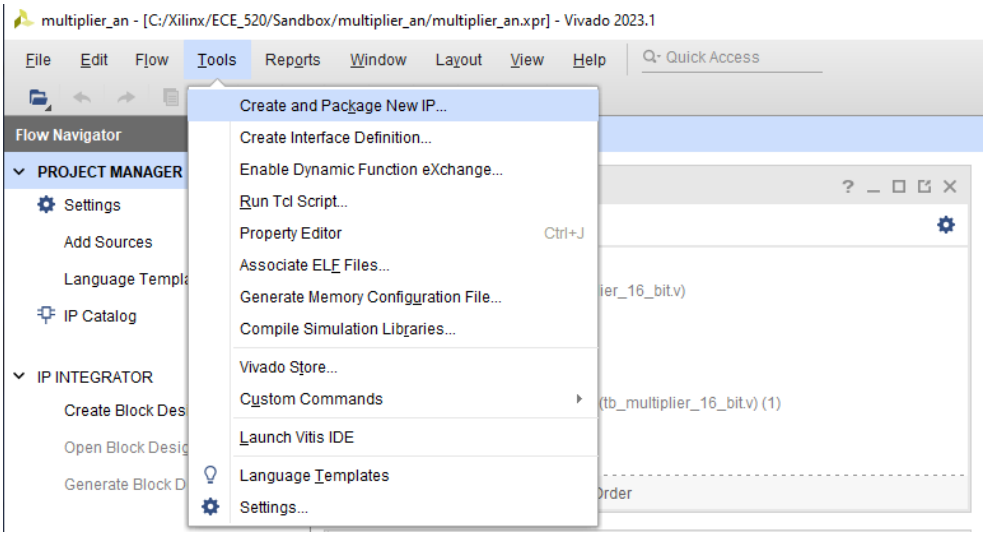
```
1   `timescale 1ns / 1ps
2
3   module multiplier_16_bit
4   (
5       input clk,
6       input rst_n,
7       input [15:0] a,
8       input [15:0] b,
9       output reg [31:0] prod
10  );
11
```

   Then, create a Verilog testbench module named **tb_multiplier_16_bit** to verify the functionality of your 16-bit multiplier. In your testbench, provide 32 pairs of values to the inputs and update them every clock cycle—this can be easily done using a for loop in Verilog.

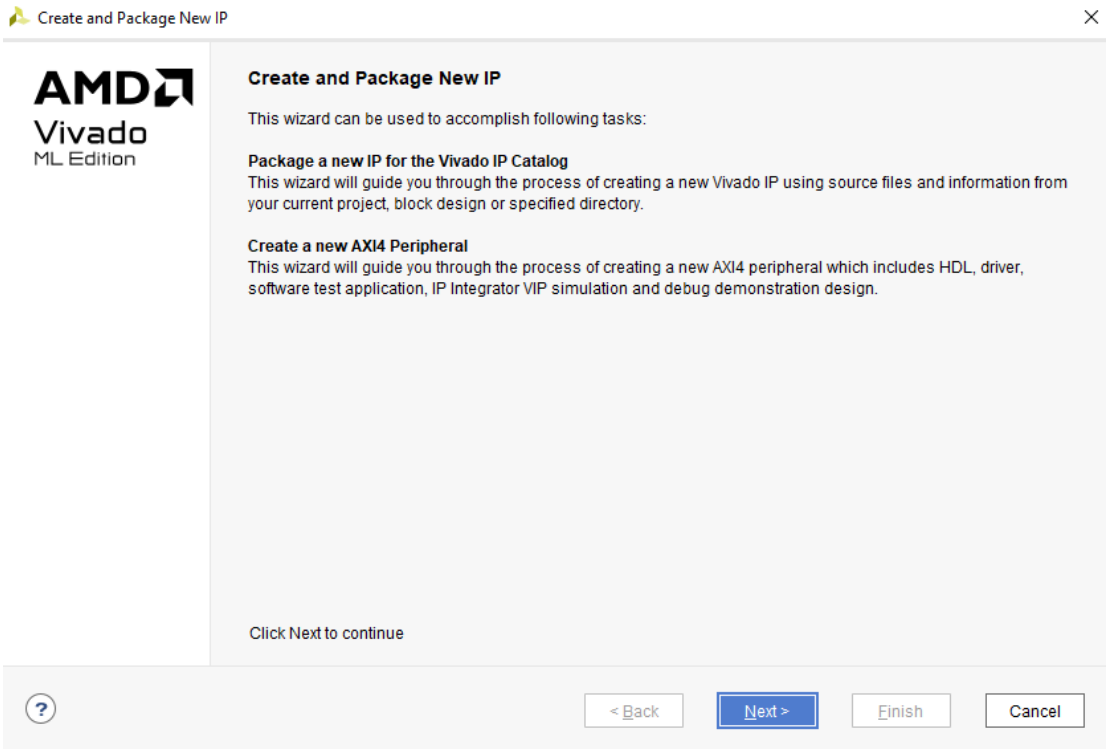
   Save a screenshot of your waveform output. It should look similar to the following waveform output.



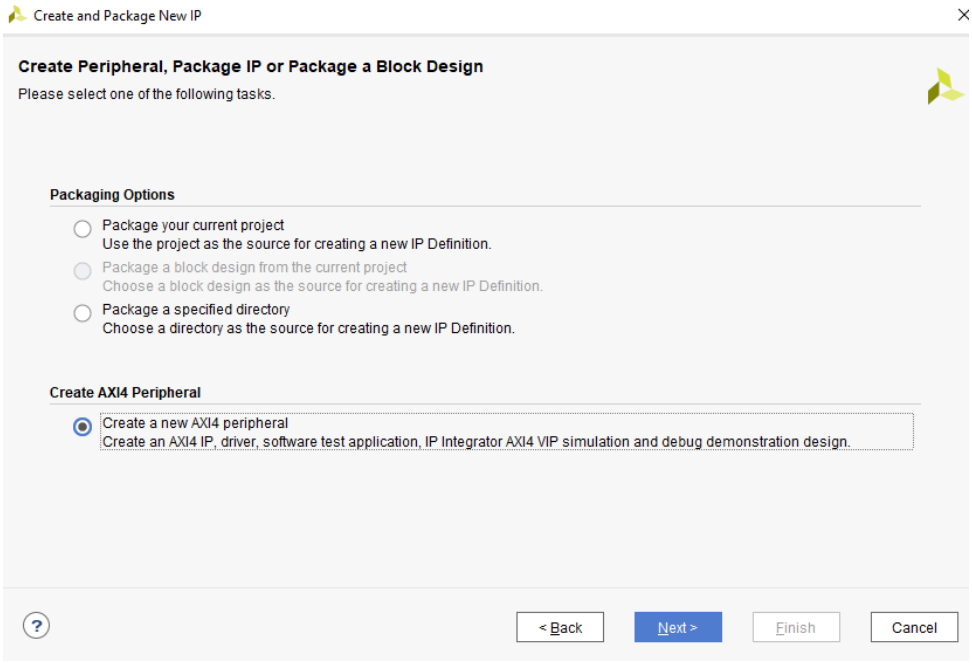
3. After verifying your 16-bit multiplier module, click on Tools and select the Create and Package New IP option.



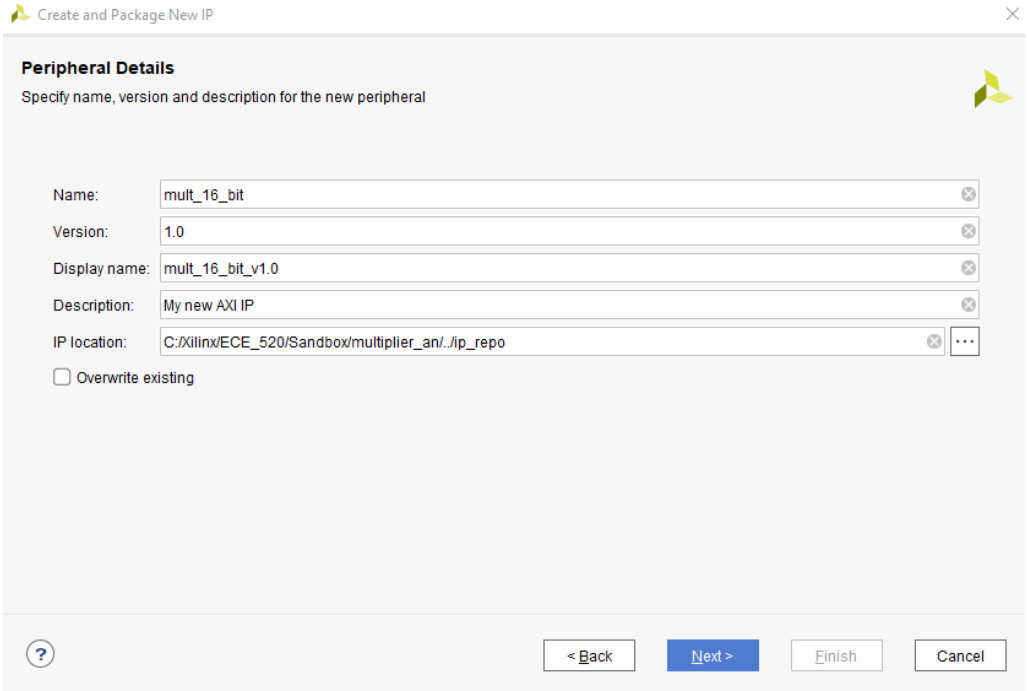
4. The following prompt should appear. Click Next.



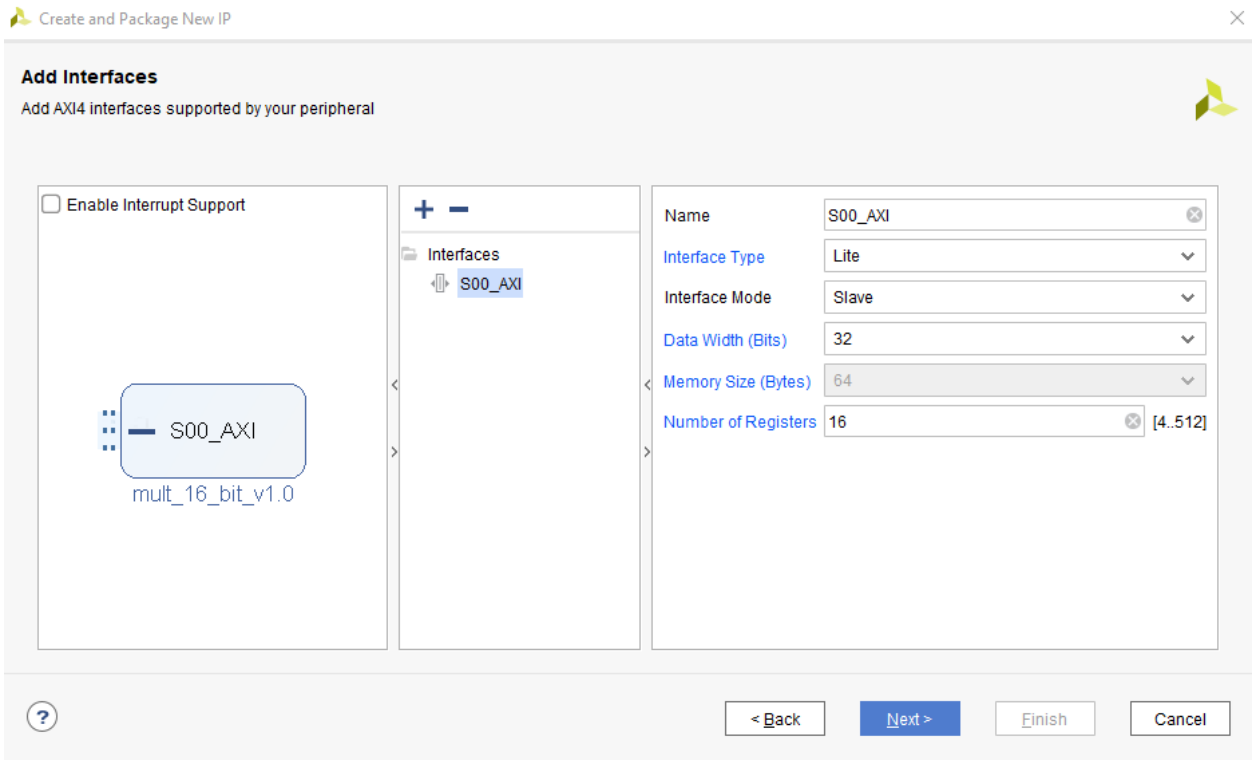
5. Select the **Create a new AXI4 peripheral** option and click Next.



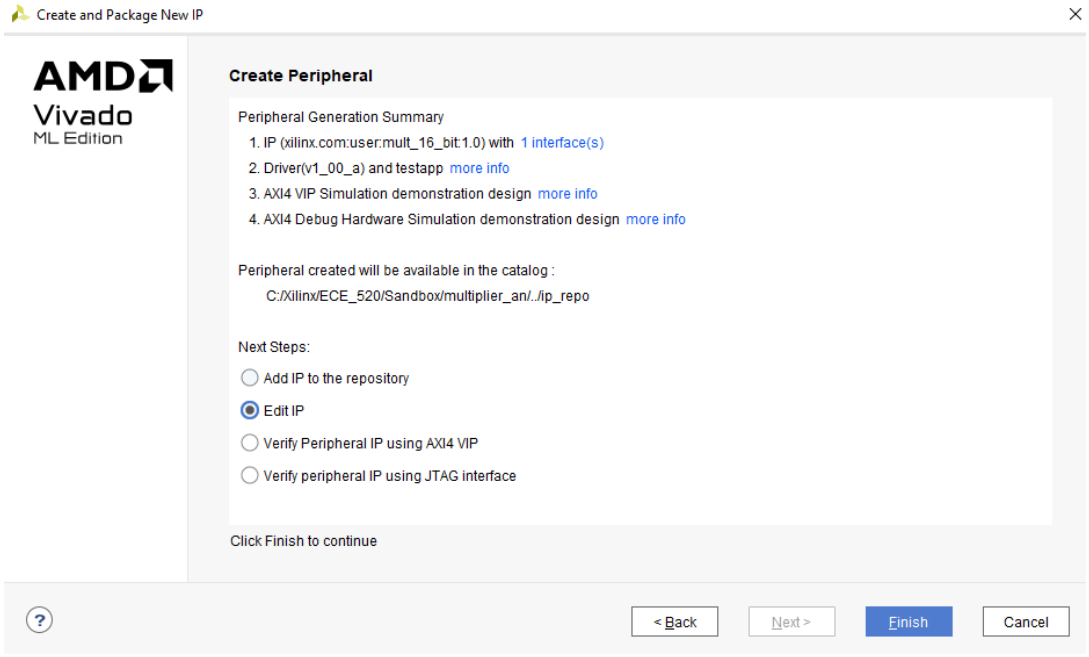
6. Update the name to **mult_16_bit** and click Next.



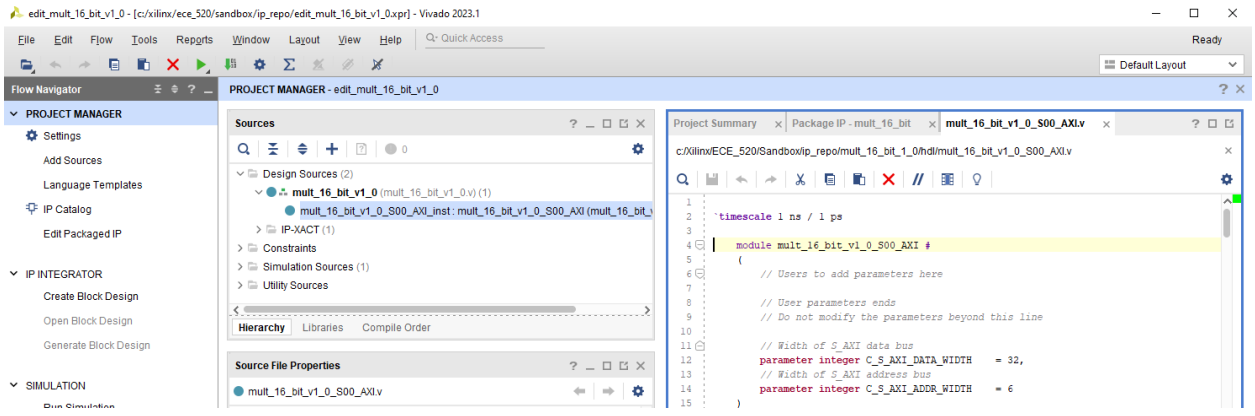
7. Configure the settings of the AXI IP to match the settings shown below. Then, click Next.
    - Name: S00_AXI
    - Interface Type: Lite
    - Interface Mode: Slave
    - Data Width (Bits): 32
    - Memory Size (Bytes): 64
    - Number of Registers: 16



8. In the next prompt, select the **Edit IP** option and click **Finish**. This will create a folder named **ip_repo**. In this example, the path of the **ip_repo** folder is located in **C:/Xilinx/ECE_520/Sandbox/ip_repo**. Make note of this path.

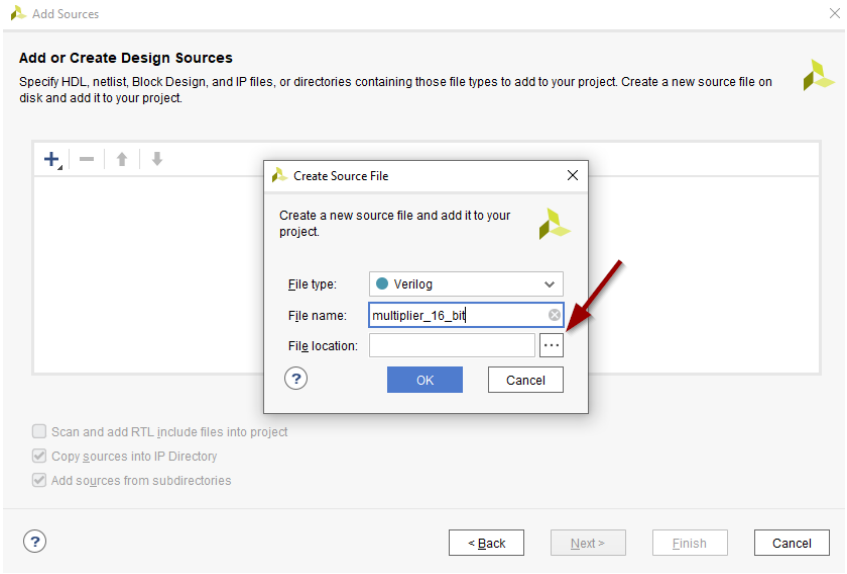


9. You should now see a new Vivado project. Double-click on the mult_16_bit_v1_0_S00_AXI.v Verilog file to open it in the text editor.

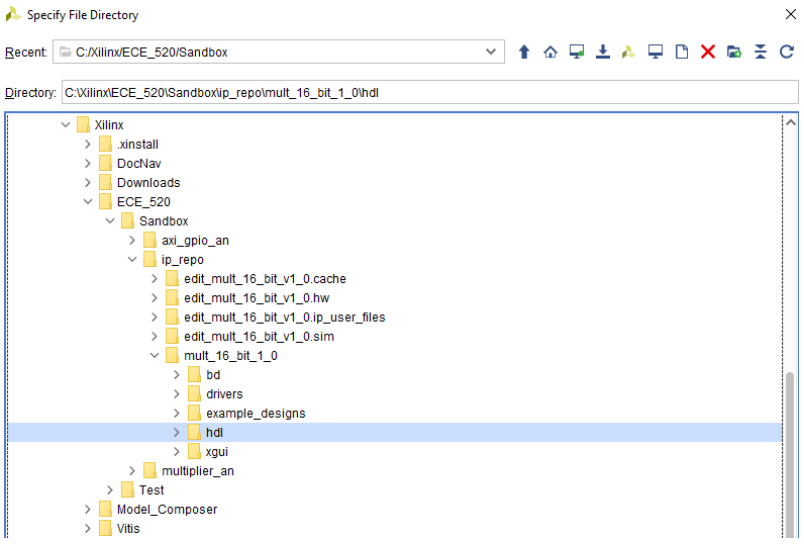


10. You will now add your 16-bit multiplier module. Click on **Add Sources** and select the **Add or create design sources** option.

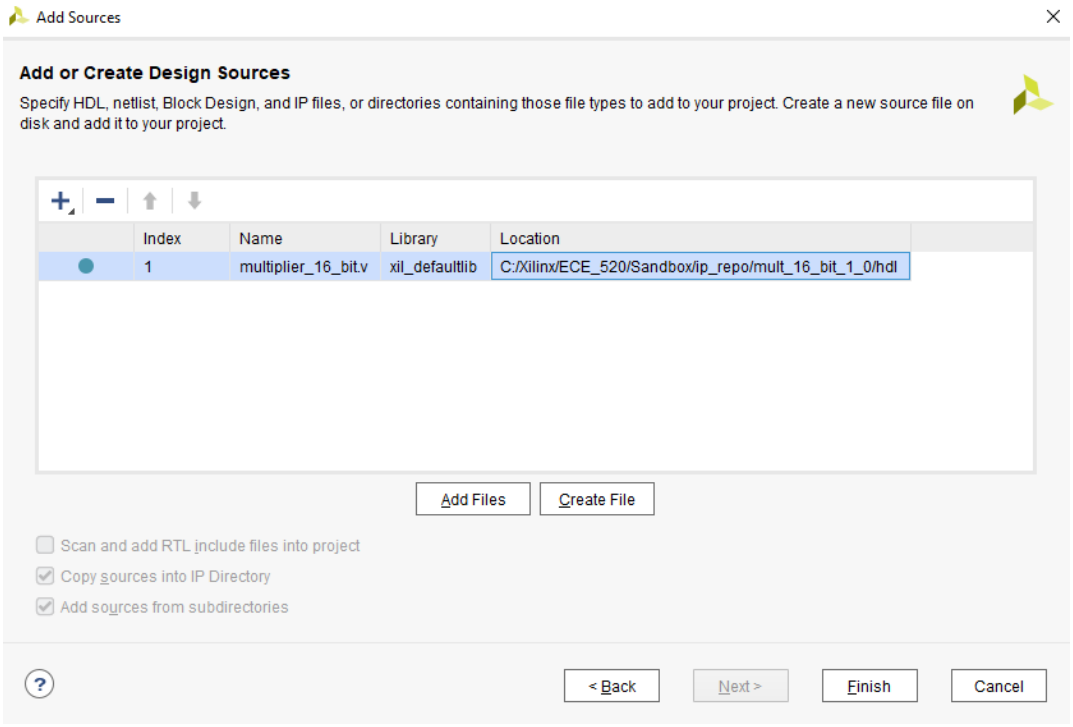
    Create a new Verilog file with the same name as your 16-bit multiplier module. Then, specify the file location by clicking on the **. . .** button as shown in the following image.



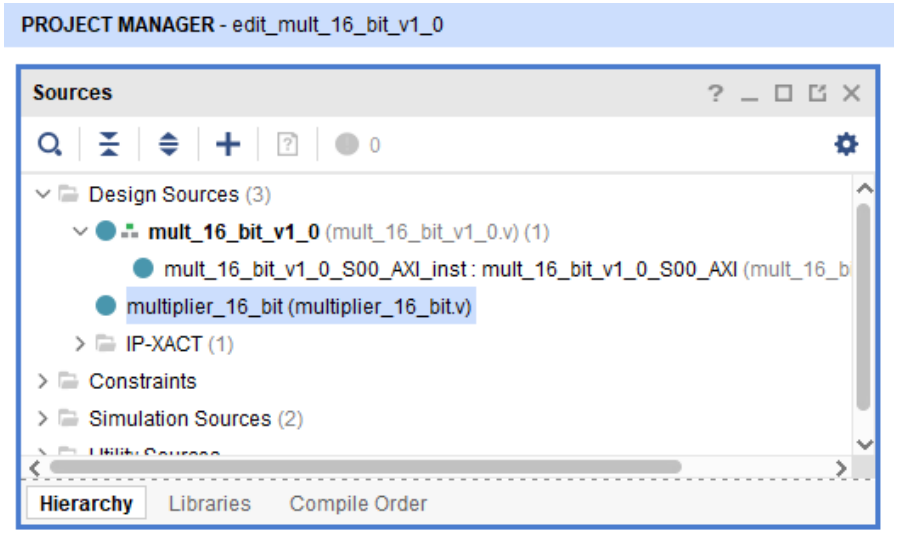
11. Specify the file location to the following path, click Select, and press OK.
    - ip_repo/mult_16_bit_1_0/hdl



12. Verify that the file location is correct and click Finish.

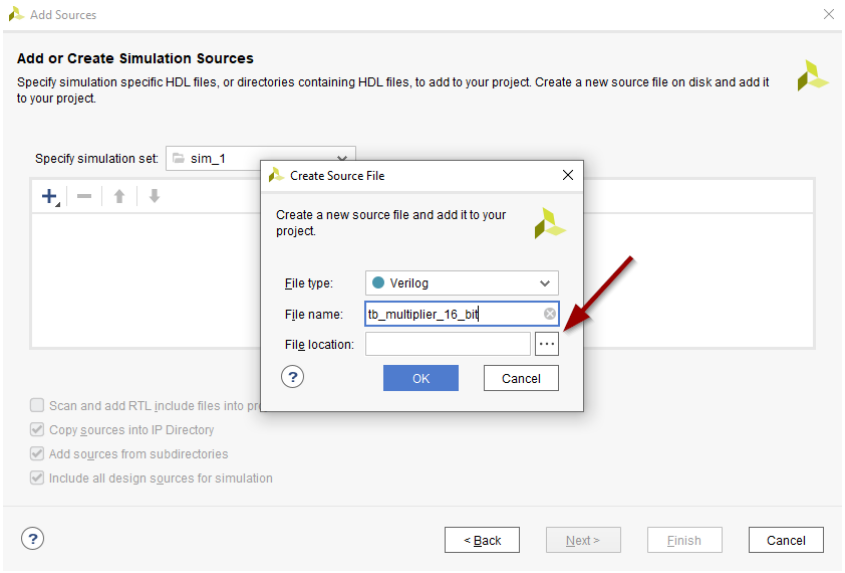


13. You should now see your **multiplier_16_bit** module under Design Sources. Paste the contents of your verified 16-bit multiplier module into this new Verilog file.

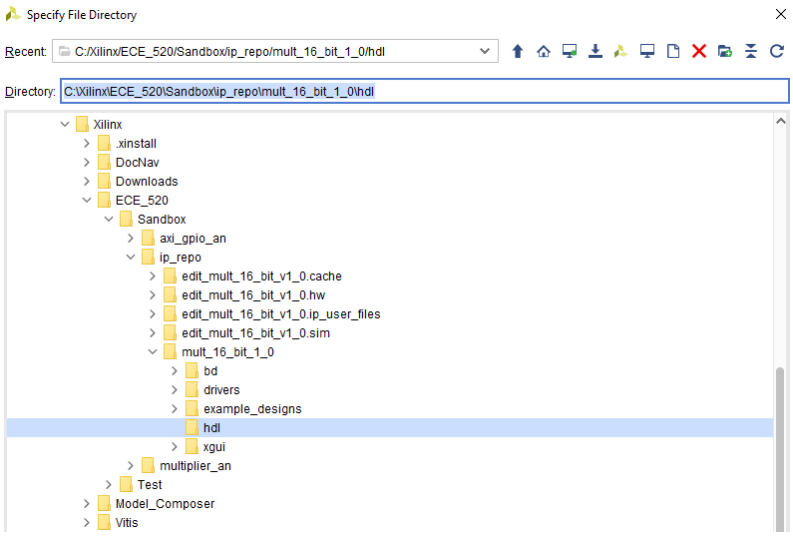


14. Next, add your 16-bit multiplier testbench module Click on **Add Sources** and select the **Add or create simulation sources** option.

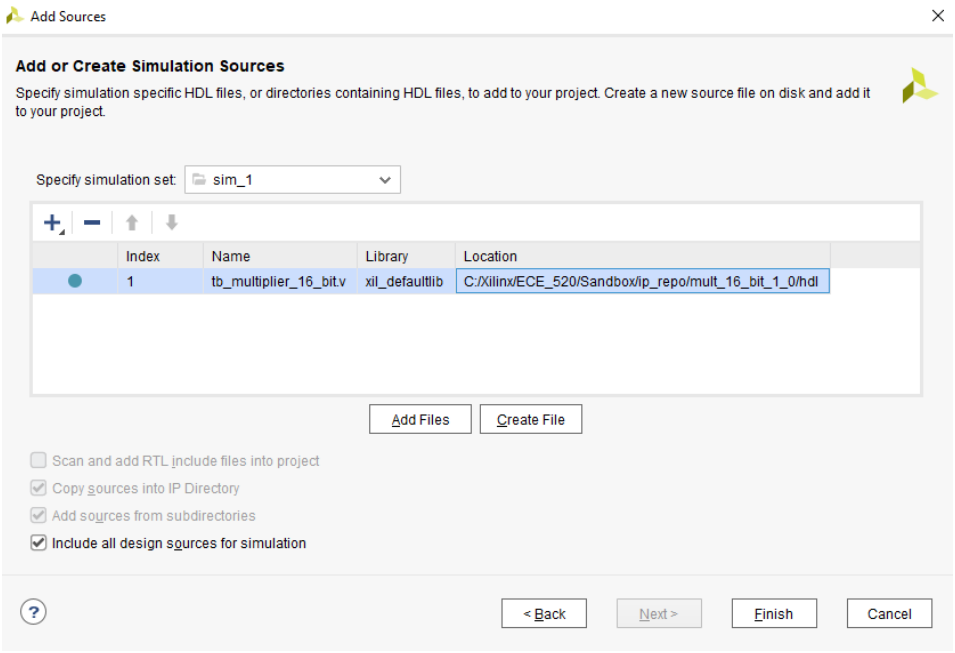
    Create a new Verilog file with the same name as your 16-bit multiplier testbench. Then, specify the file location by clicking on the **. . .** button as shown in the following image.



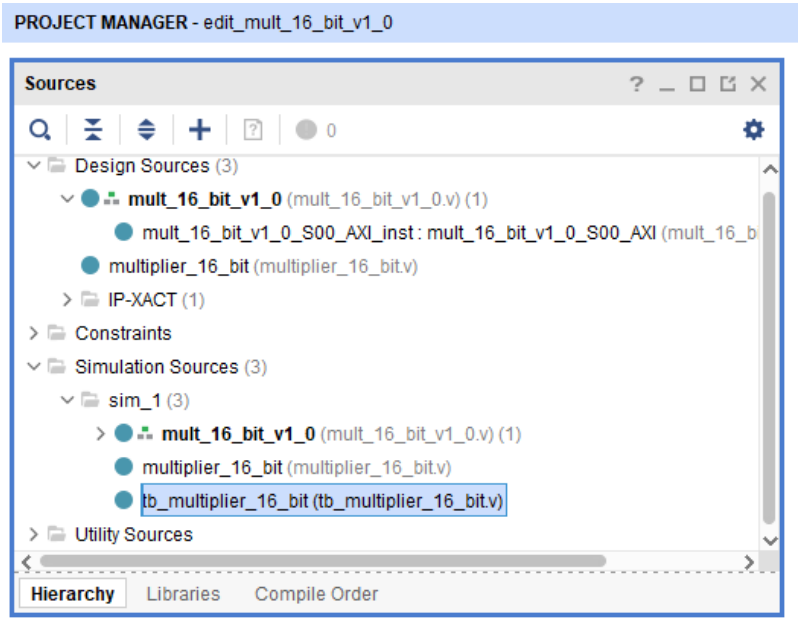
15. Specify the file location to the following path, click Select, and press OK.
    - ip_repo/mult_16_bit_1_0/hdl



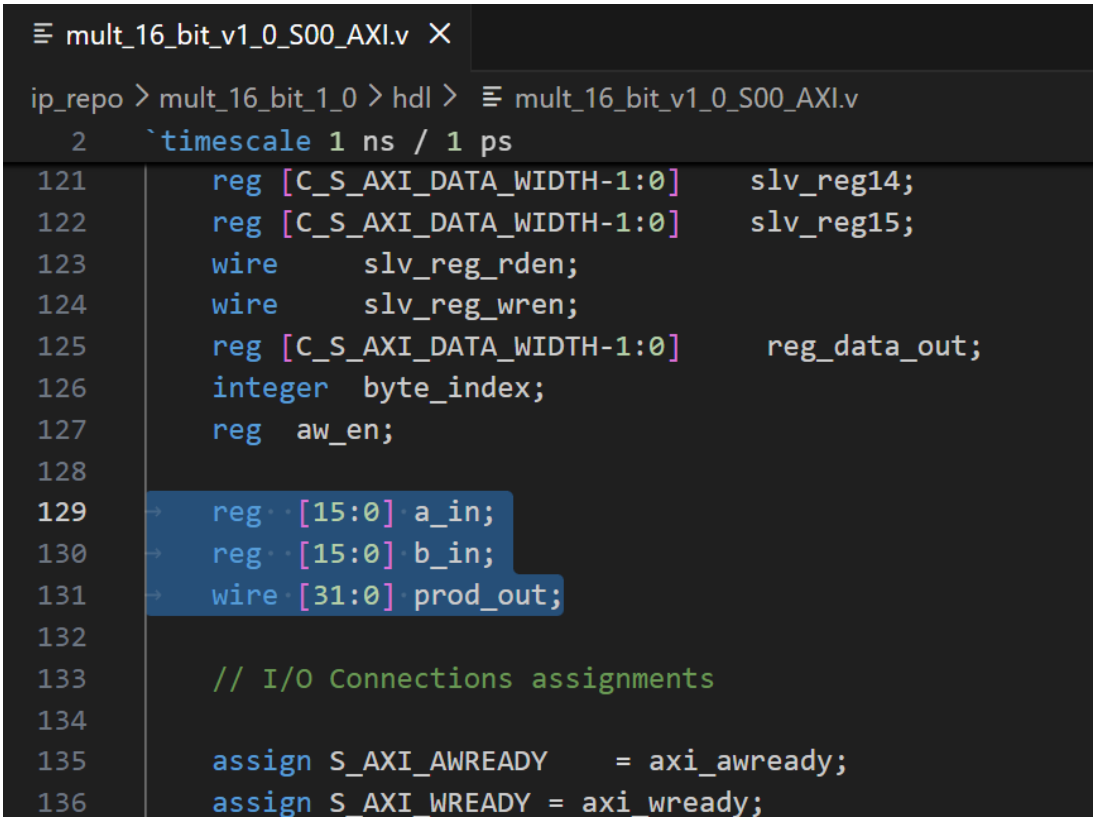
16. Verify that the file location is correct and click Finish.



17. You should now see your **tb_multiplier_16_bit** testbench module under Simulation Sources. Paste the contents of your testbench into this new Verilog file.



18. In the **mult_16_bit_v1_0_S00_AXI** module, add the following lines. These will be used to make connections to the 16-bit multiplier module.



19. Next, add the following always block in the same module. This will map the following registers:
    - Lower 16 bits of the AXI register at offset 0x00 → a_in
    - Lower 16 bits of the AXI register at offset 0x04 → b_in

mult_16_bit_v1_0_S00_AXI.v

ip_repo > mult_16_bit_1_0 > hdl > mult_16_bit_v1_0_S00_AXI.v

```
  2   `timescale 1 ns / 1 ps
519       begin
524         else
525           begin
529             if (slv_reg_rden)
533           end
534       end
535
536       // Add user logic here
537
538       always @(posedge S_AXI_ACLK) begin
539           if (S_AXI_ARESETN == 1'b0) begin
540               a_in <= 0;
541               b_in <= 0;
542           end else begin
543               a_in <= slv_reg0[15:0];
544               b_in <= slv_reg1[15:0];
545           end
546       end
547
548       // User logic ends
549
550       endmodule
551
552
```

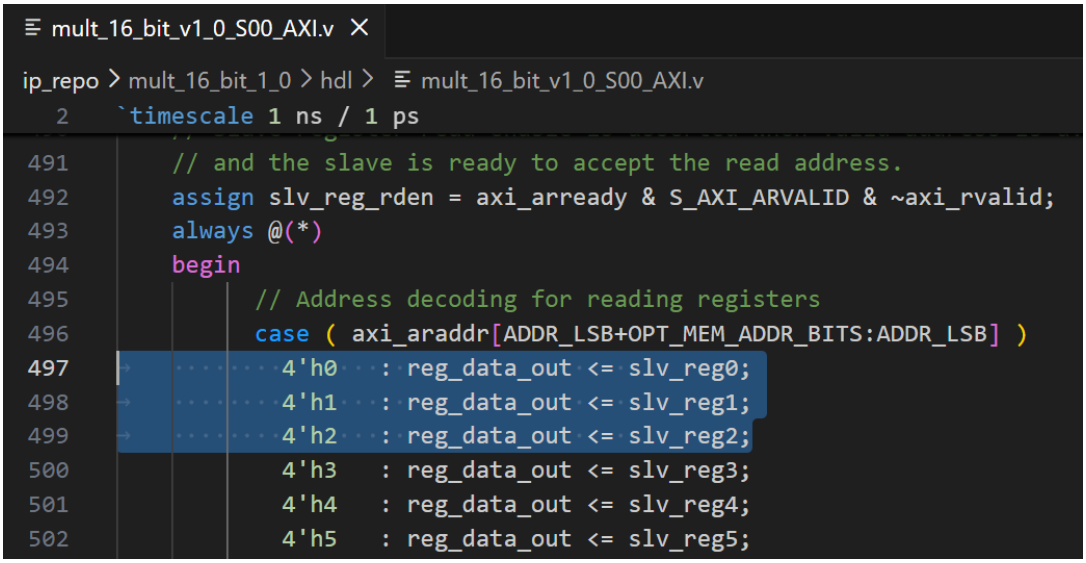
20. Instantiate your 16-bit multiplier module as shown in the image below.

mult_16_bit_v1_0_S00_AXI.v

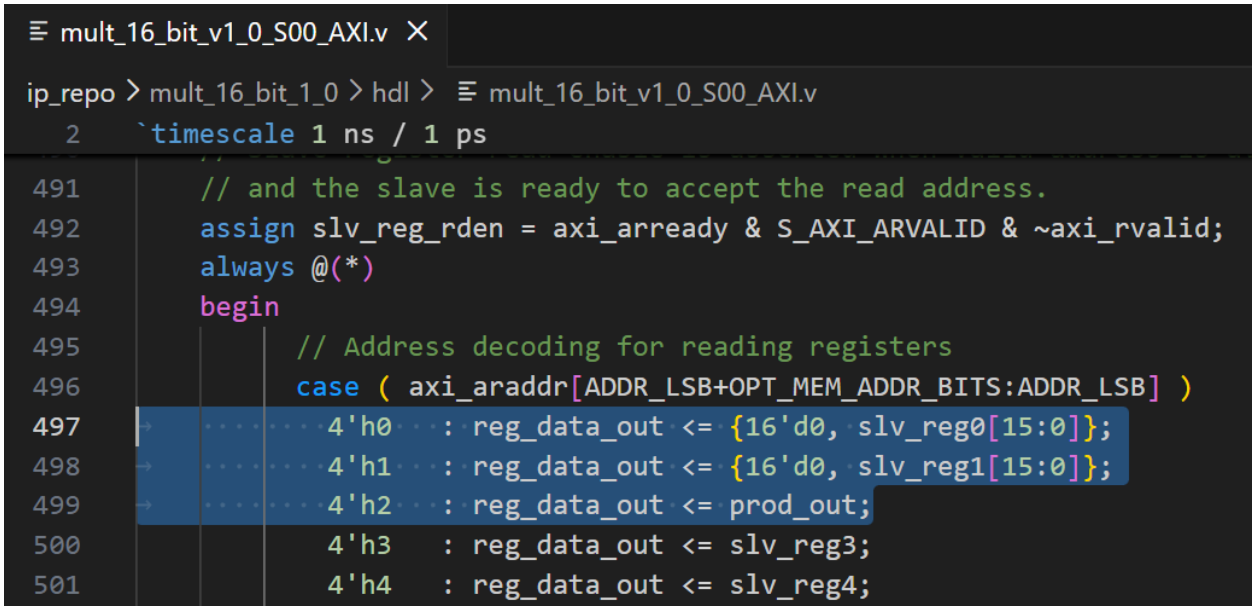
ip_repo > mult_16_bit_1_0 > hdl > mult_16_bit_v1_0_S00_AXI.v

```
2   `timescale 1 ns / 1 ps
535
536     // Add user logic here
537
538     always @(posedge S_AXI_ACLK) begin
539         if (S_AXI_ARESETN == 1'b0) begin
540             a_in <= 0;
541             b_in <= 0;
542         end else begin
543             a_in <= slv_reg0[15:0];
544             b_in <= slv_reg1[15:0];
545         end
546     end
547
548     multiplier_16_bit multiplier_16_bit_inst_
549     (
550         .clk(S_AXI_ACLK),
551         .rst_n(S_AXI_ARESETN),
552         .a(a_in),
553         .b(b_in),
554         .prod(prod_out)
555     );
556
557     // User logic ends
558
559     endmodule
560
```

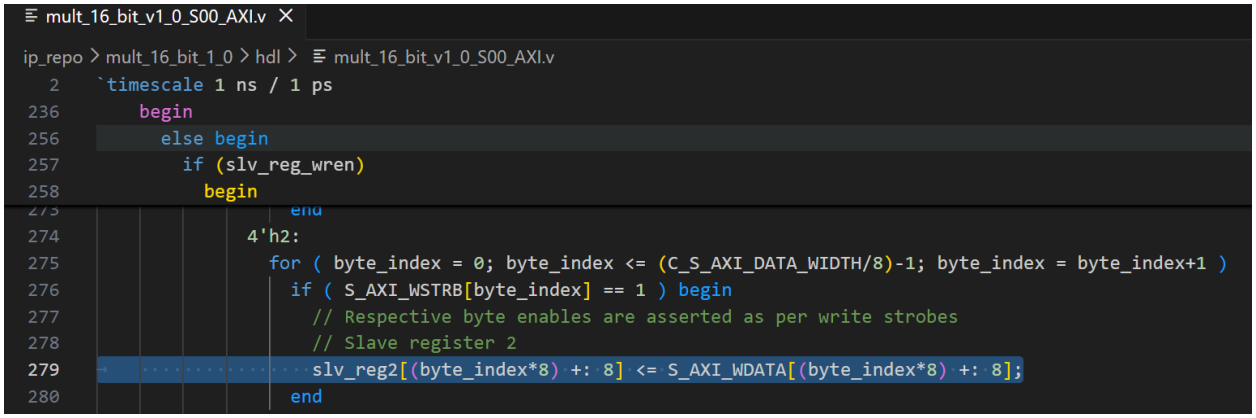
21. Next, look for the following highlighted lines.



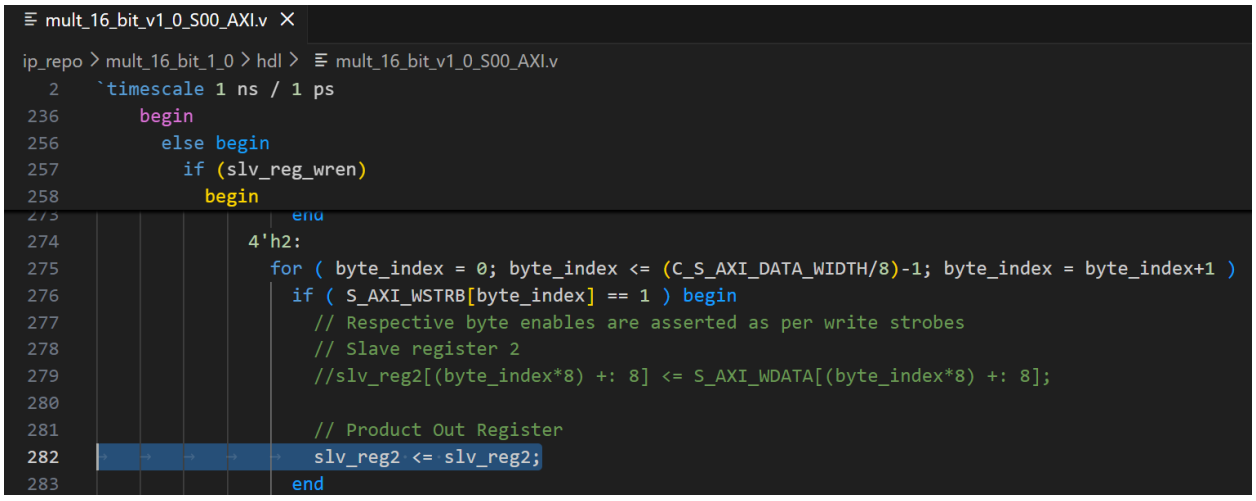
22. Then, update the highlighted lines as shown in the image below. The upper 16 bits of slv_reg0 (offset 0x00) and slv_reg1 (offset 0x04) will be 0. In addition, slv_reg2 (offset 0x08) will display the product from the 16-bit multiplier module.



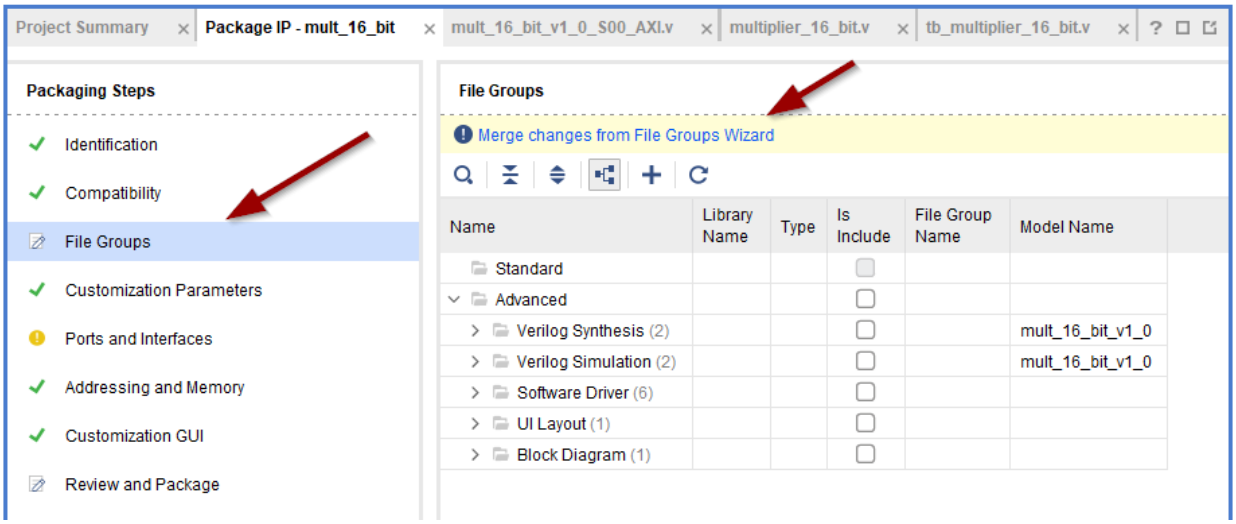
23. Look for the block of code that starts with **4'h2**.



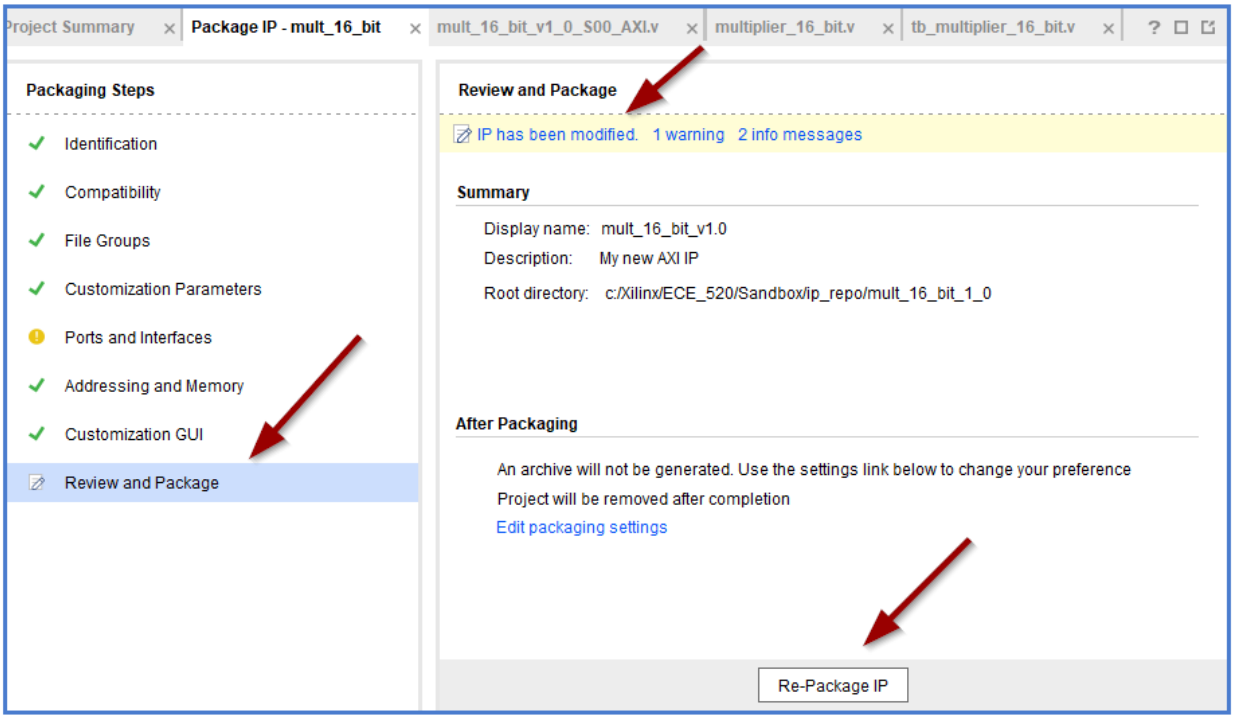
24. Replace the following line with the one highlighted in the image below. This ensures that the register used to hold the product from the 16-bit multiplier module will be read-only. Save your changes.



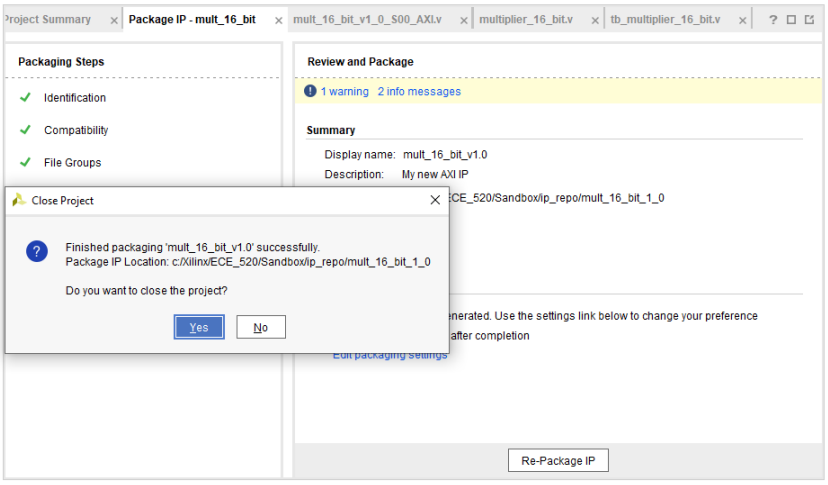
25. In Vivado, click on the **Package IP – mult_16_bit** tab and select **File Groups**. Then, click on the **Merge changes from File Groups Wizard** option.



26. Next, select **Review and Package** and click on **IP has been modified**. Then, click on the **Re-Package IP** button.

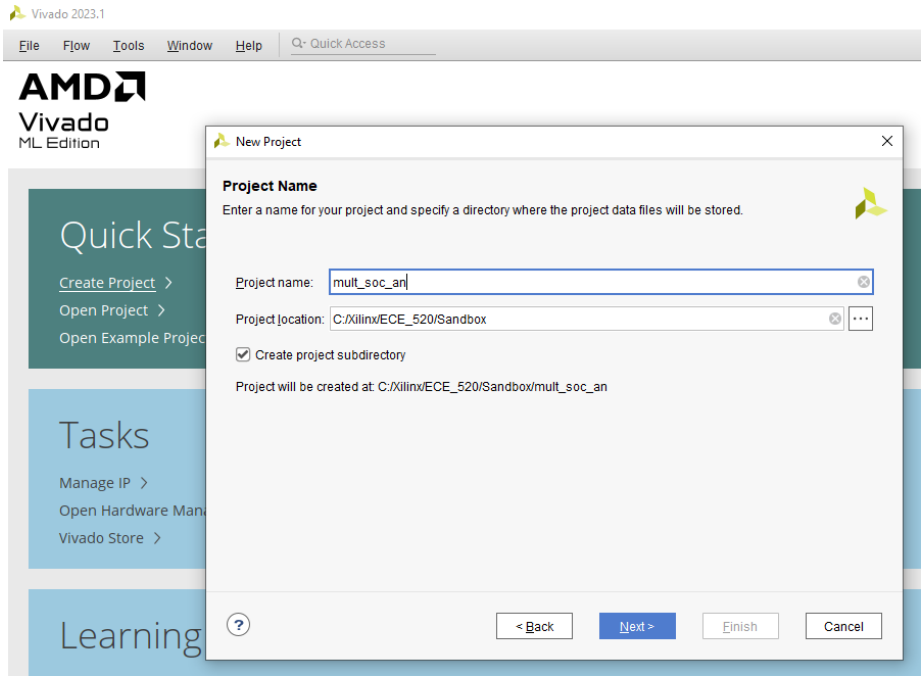


27. Next, you will be prompted to close the project. Close the current Vivado project by clicking Yes.

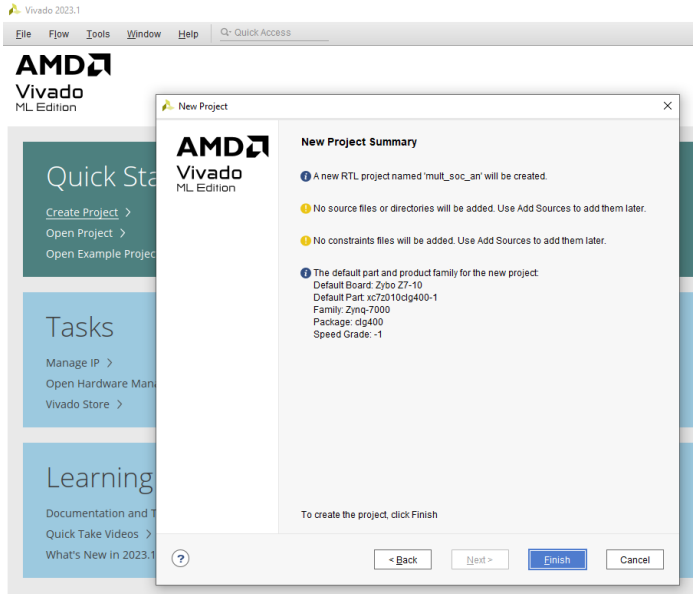


28. Relaunch Vivado and create another new project called **mult_soc_<your_initials>**.

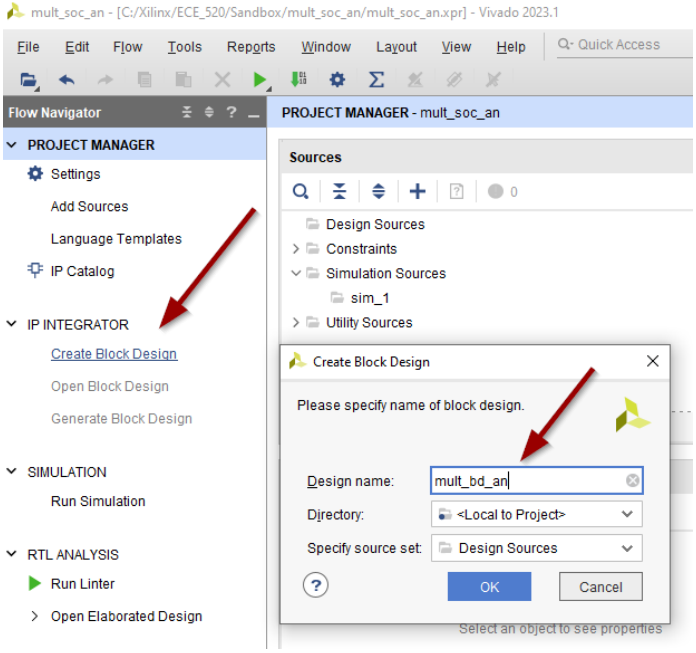
    An example is shown below. Hit the Next button after you've entered the project name and the project location.



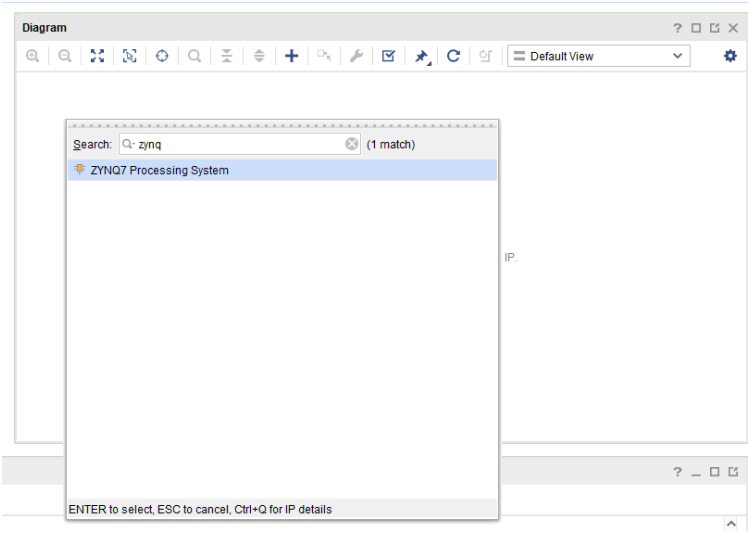
29. Configure the Vivado project settings and then review the details in the **New Project Summary**. Click Finish.



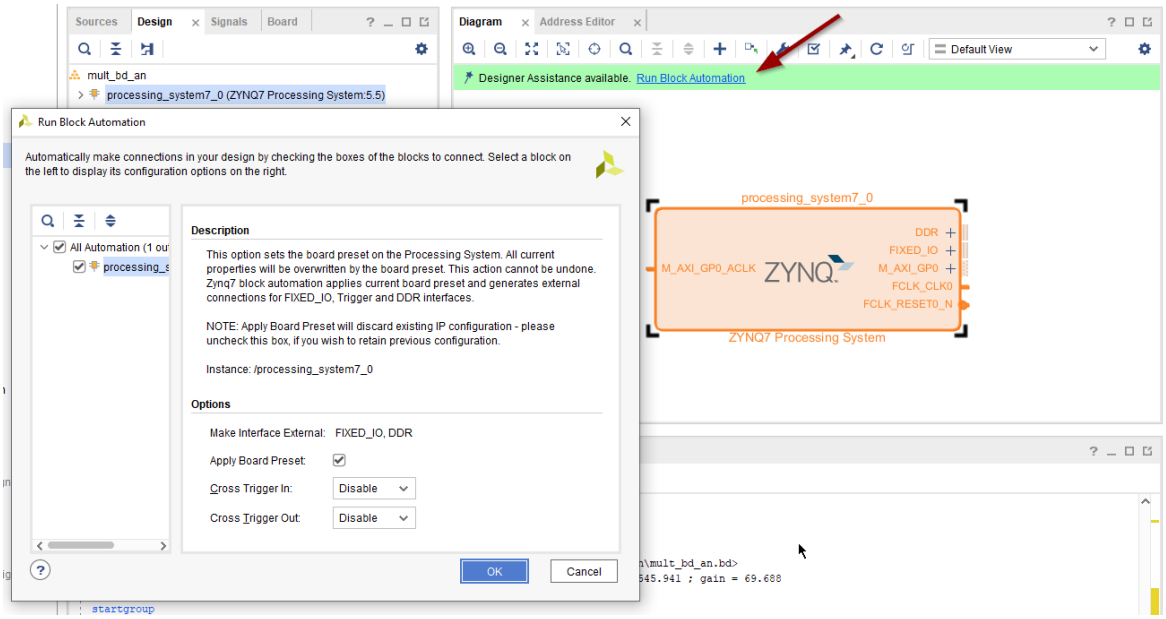
30. The **Project Manager** window should now appear. Under IP Integrator, click **Create Block Design** and enter **mult_bd_<your_initials>** for the design name. Hit OK.



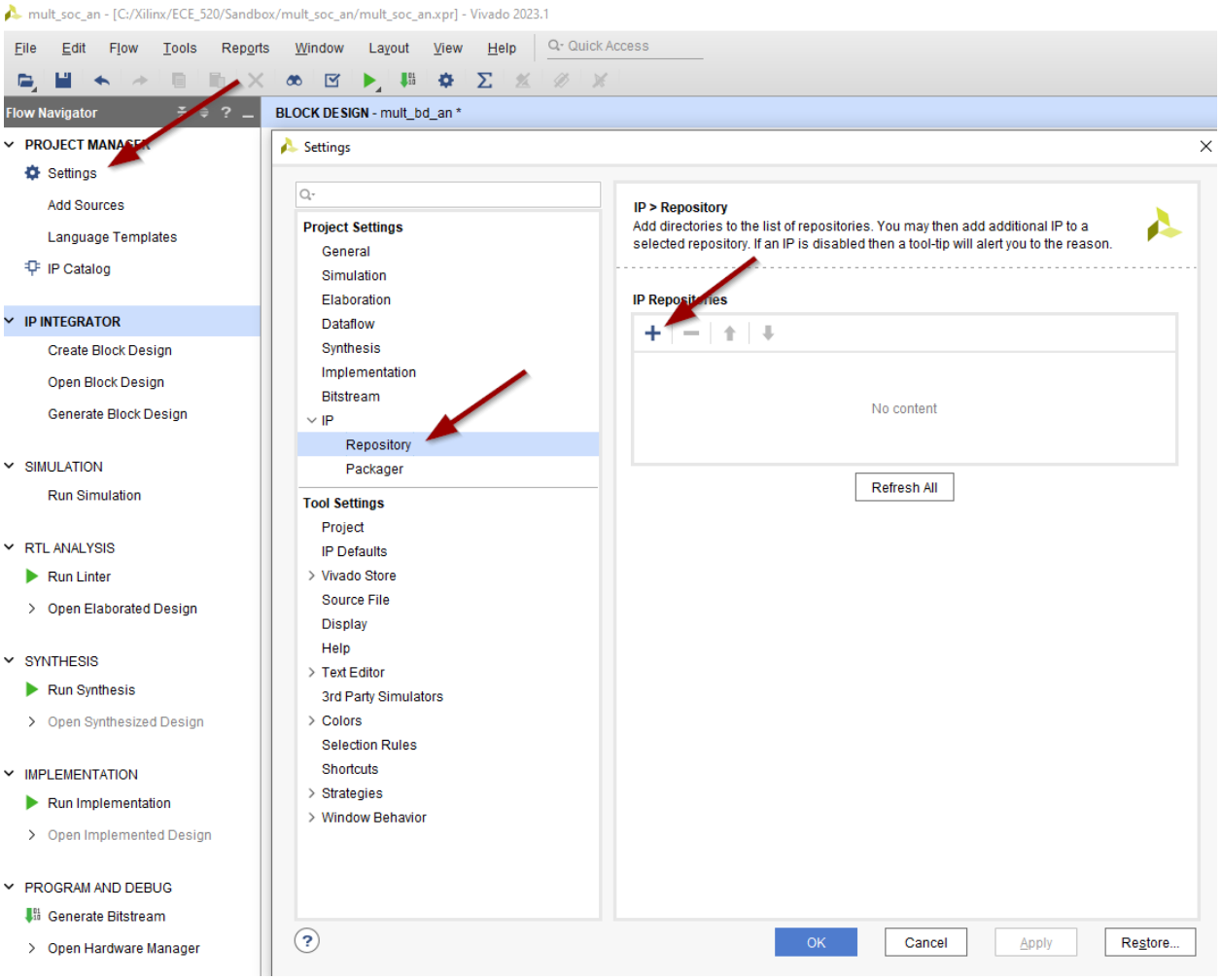
31. The **Block Design** window should now appear. Click on the **+** button and type **zynq** in the search field. Double-click on the **ZYNQ7 Processing System** option. This will instantiate the Zynq PS block in the block design canvas.



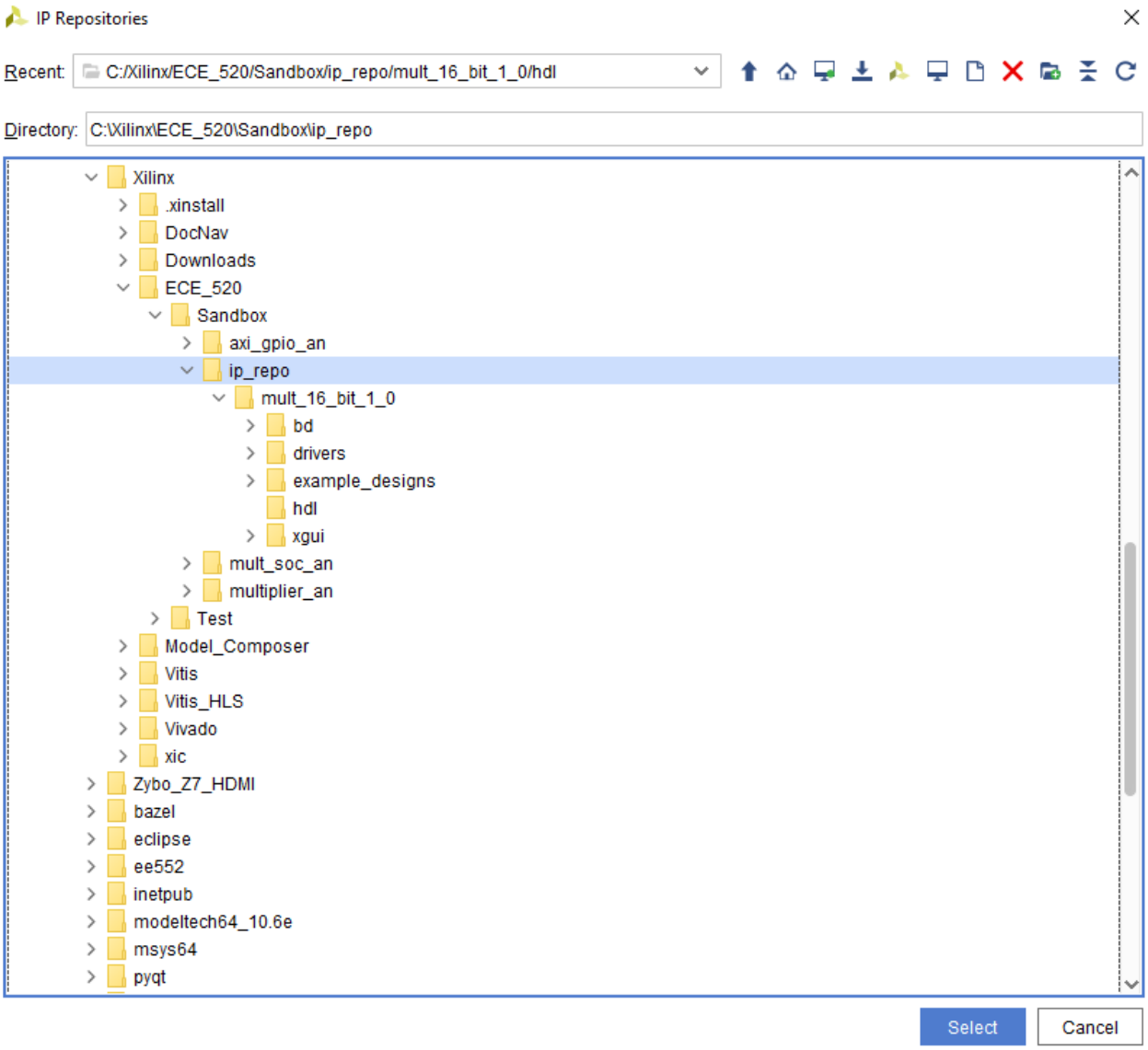
32. Click **Run Block Automation** and do not modify the default options. Hit OK.



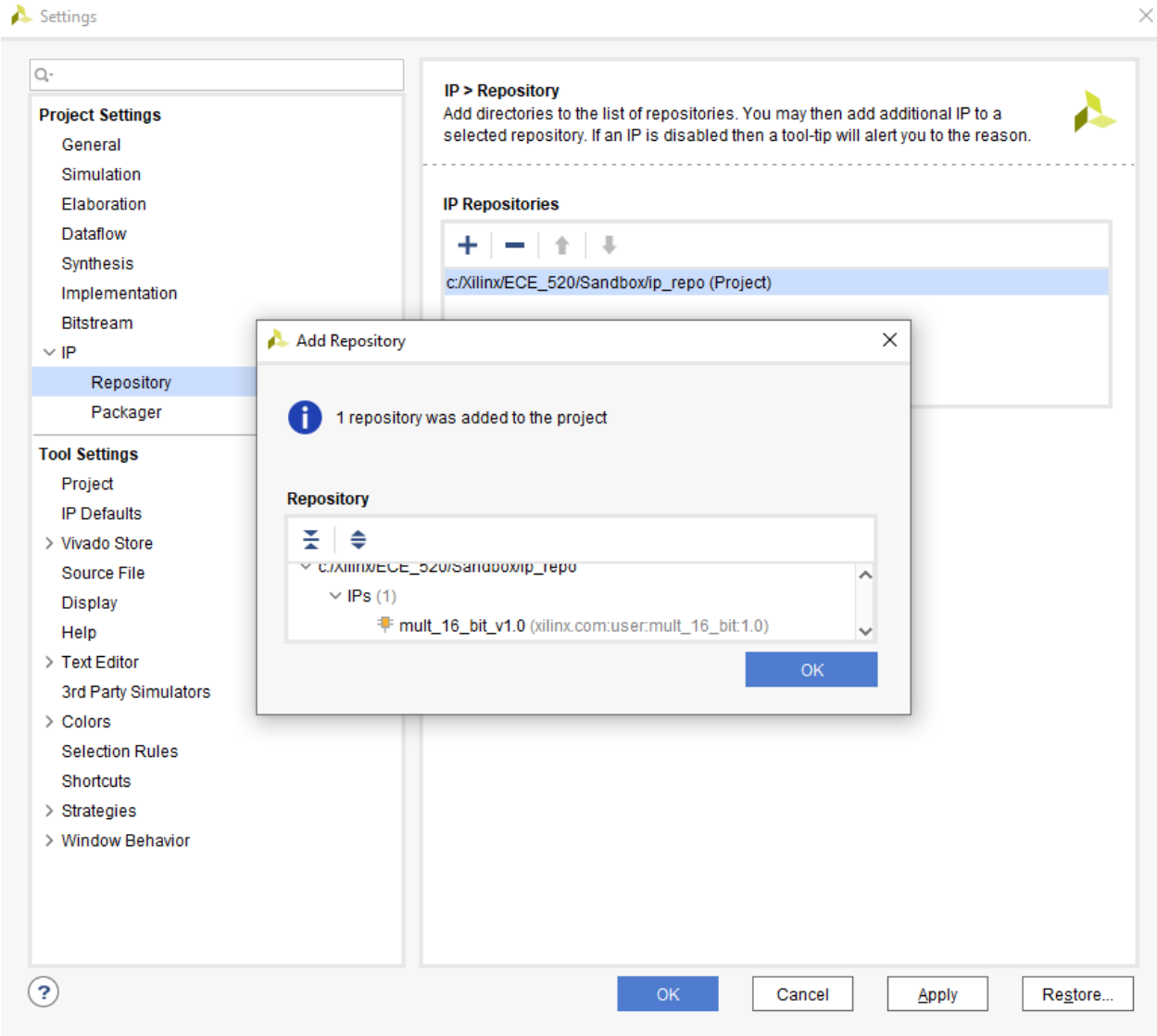
33. You will now add your custom 16-bit multiplier AXI IP. Click on **Settings**. Under **IP**, select the **Repository** option and then click the **+** button.



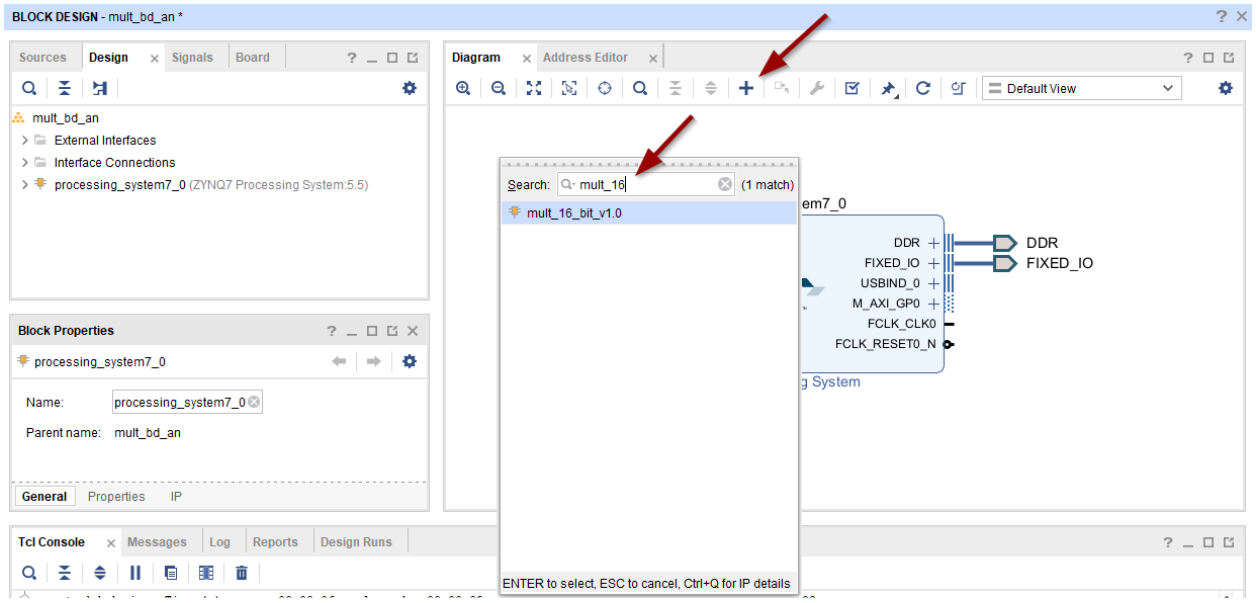
34. Select the **ip_repo** folder that was created earlier.



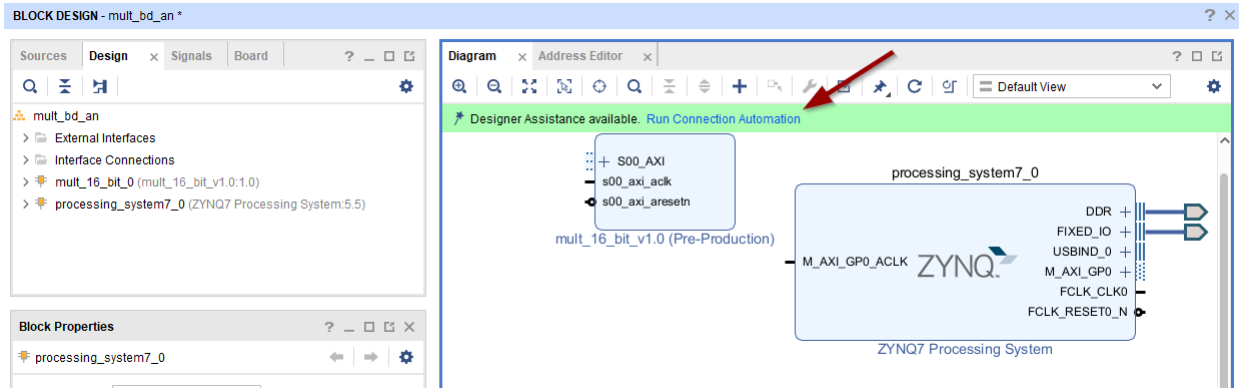
35. You should now see the following prompt that verifies that you've added the 16-bit multiplier AXI IP. Hit OK.



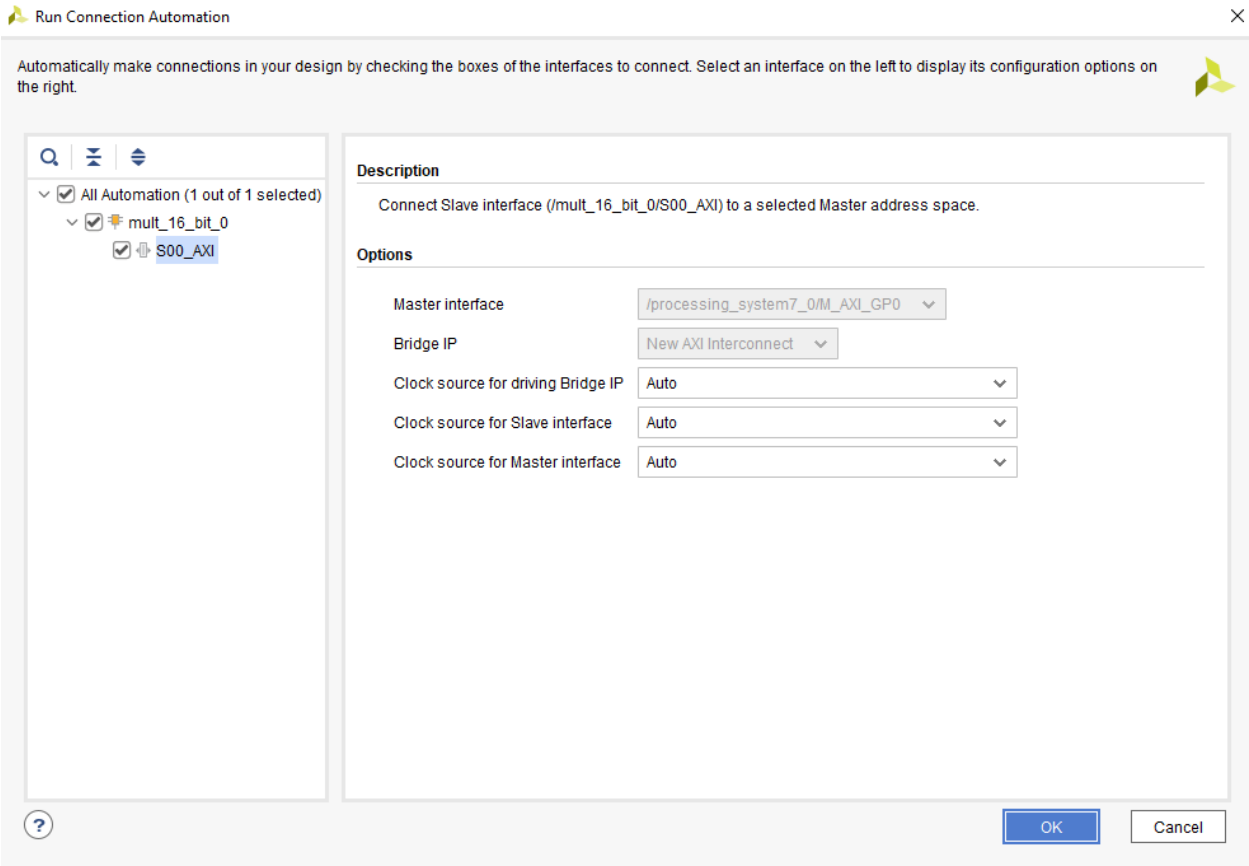
36. Click on the + button again and type **mult_16**. You should see your custom AXI IP. Double-click on it to instantiate your IP in the block design canvas.



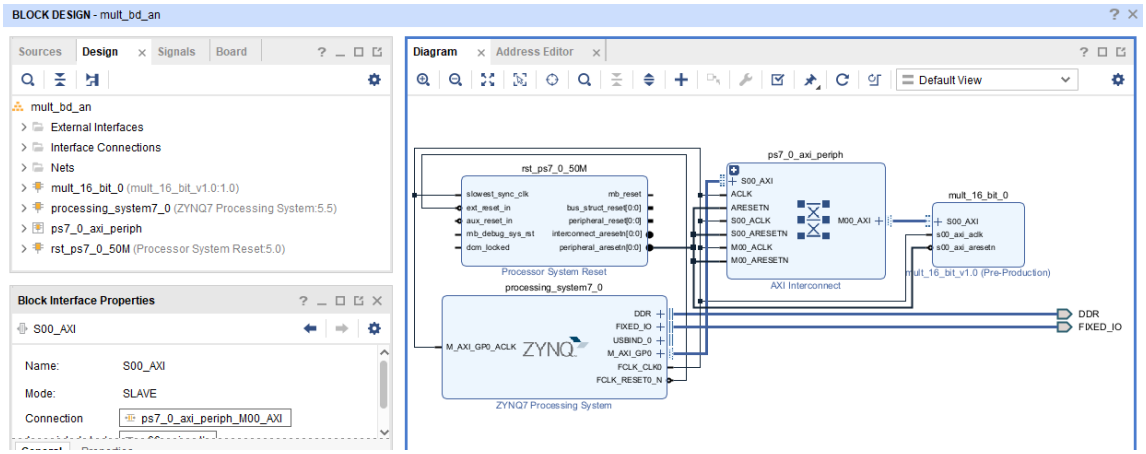
37. Click the **Run Connection Automation** option.



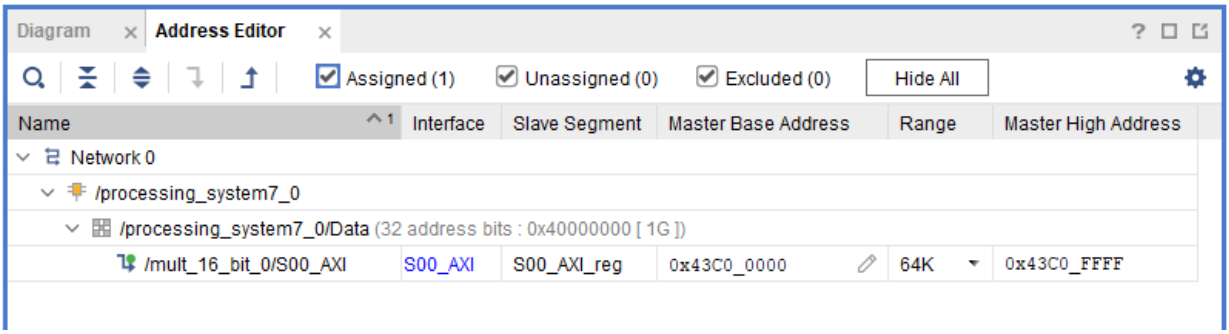
38. Select all of the options under **All Automation**. Then, click the OK button.



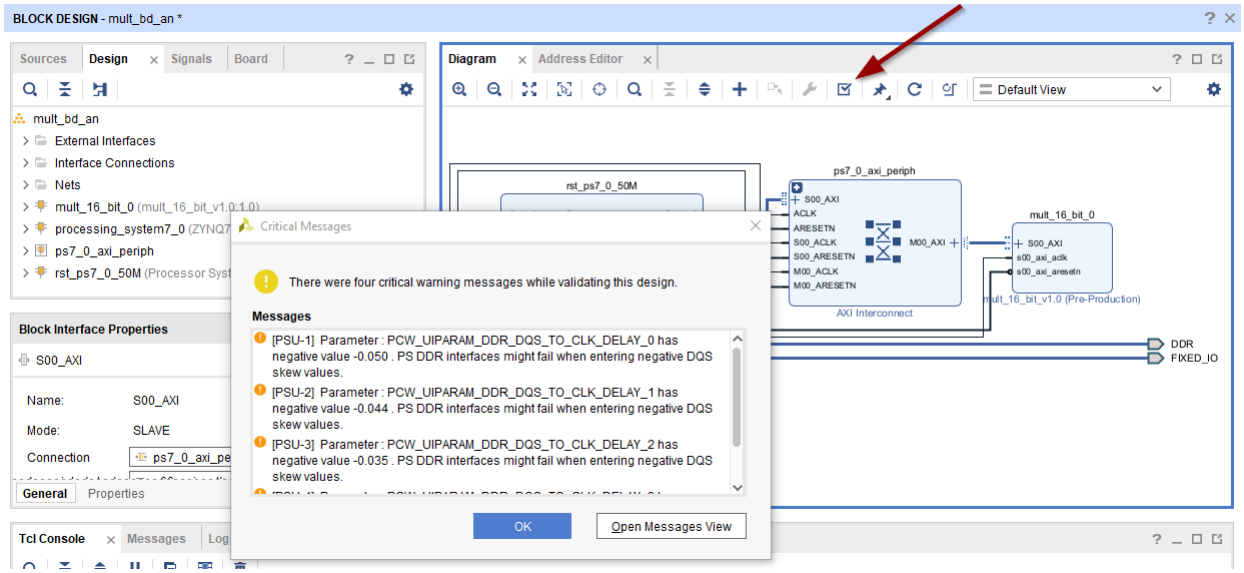
39. You should now see a similar block design diagram. Vivado has automatically placed the connections for you.



40. Click on the **Address Editor** tab. Make note of the address assigned to your custom IP.

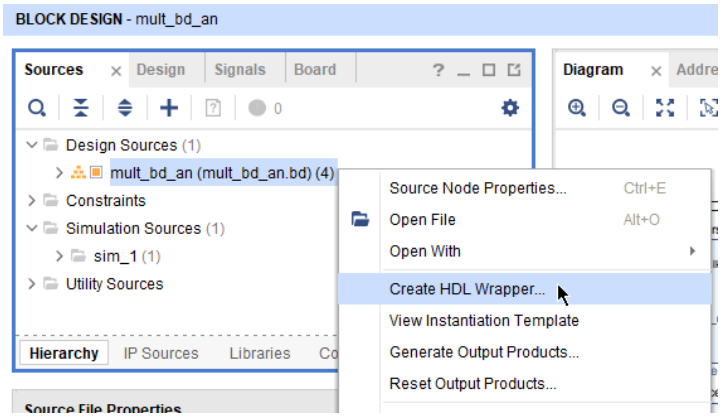


41. Go back to the **Diagram** page and click on the **Validate Design** button. After validating the design, some warnings might appear, but these warnings can be ignored.

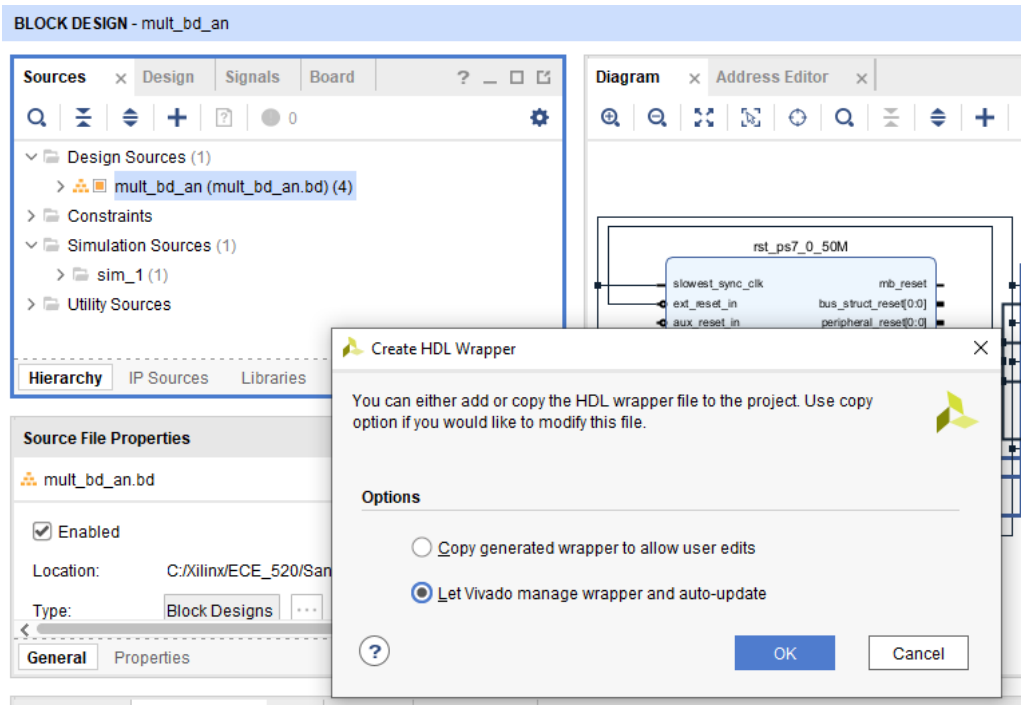


42. After the design has been validated, go to **Sources** and expand **Design Sources**.

    Right-click on the **mult_bd_<your_initials>.bd** file and select the **Create HDL Wrapper** option.

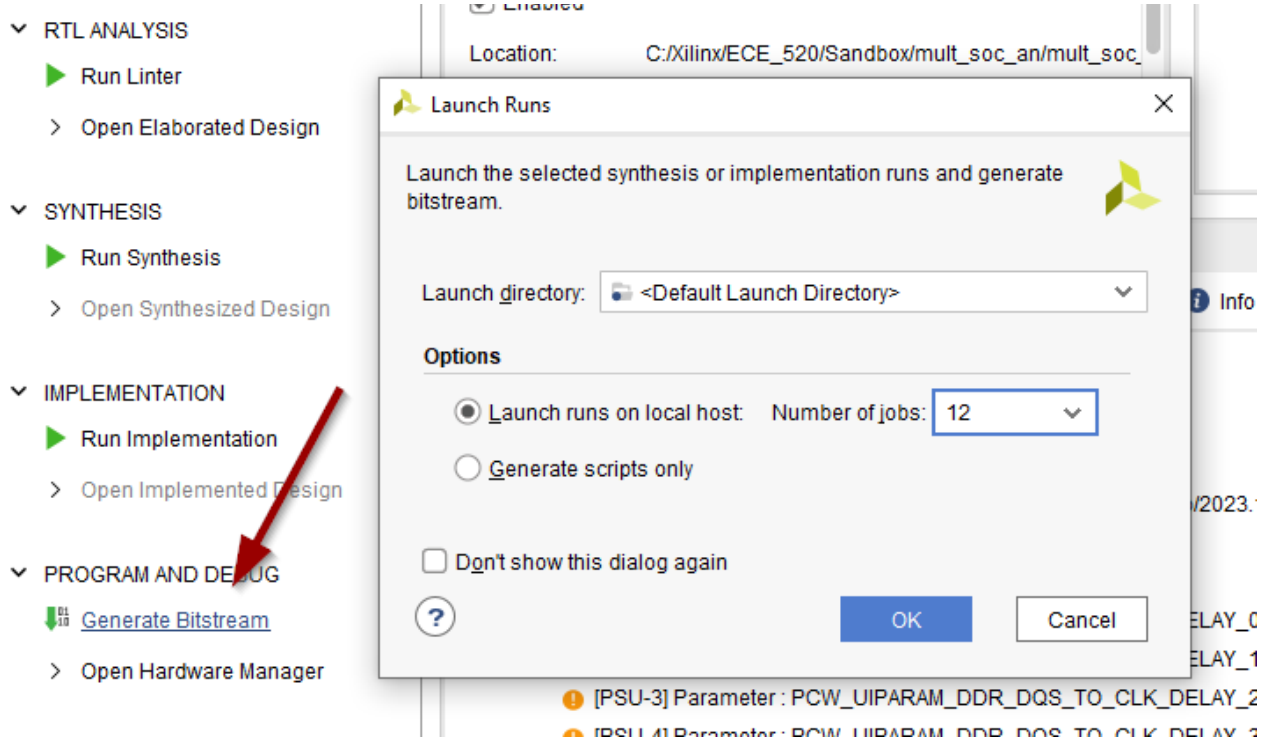


43. Then, select the **Let Vivado manage wrapper and auto-update** option and hit OK.

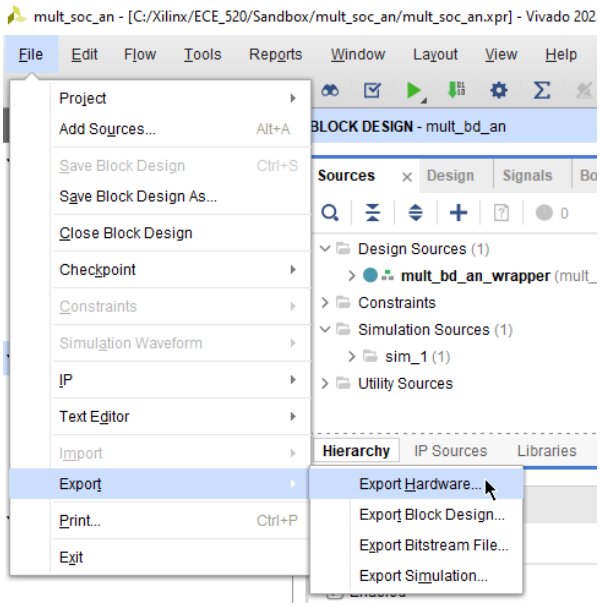


44. After the HDL wrapper has been generated, save the Vivado project.

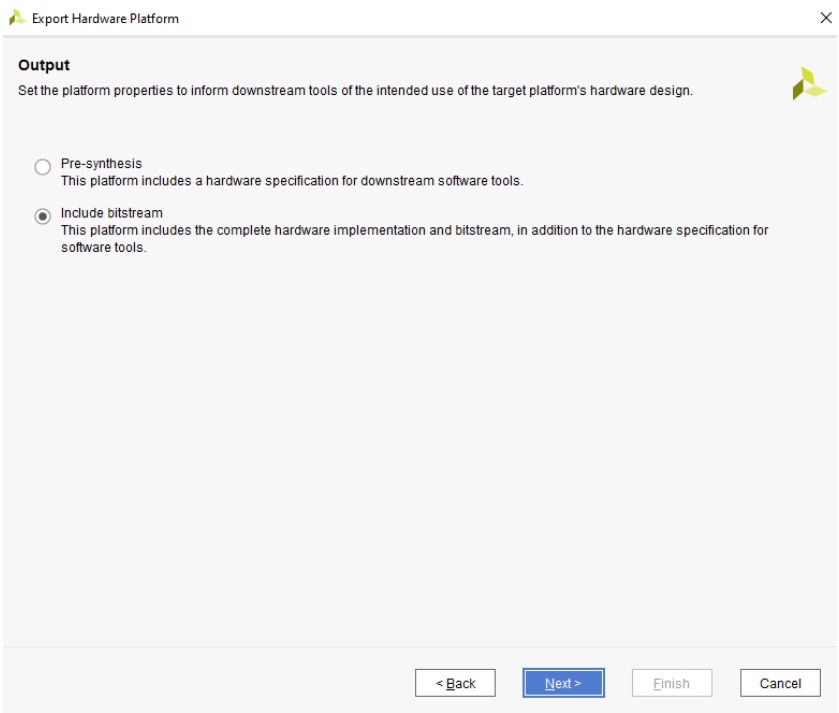
Next, click on the **Generate Bitstream** option. This will run Synthesis and then Implementation if synthesis is successful.



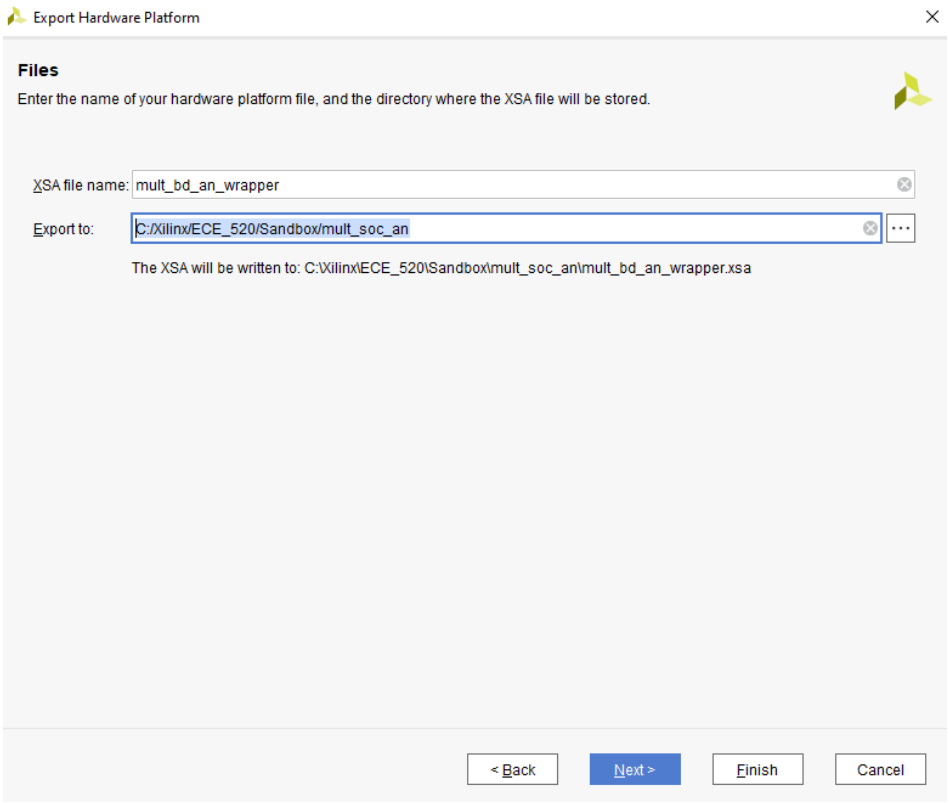
45. After the bitstream has successfully generated, click on **File > Export > Export Hardware**. The Export Hardware Platform window will appear.



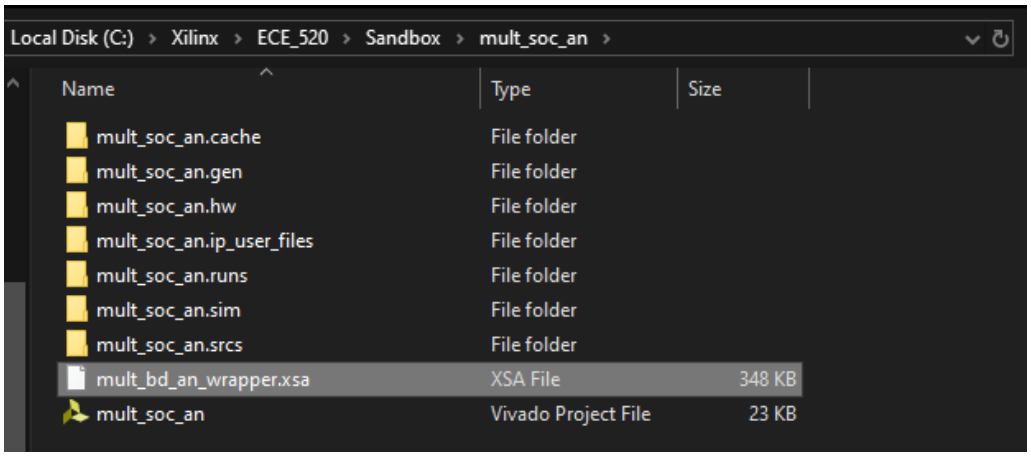
46. In the **Output** page, select the **Include bitstream** option and click **Next**.



47. Make note of the location where the **.xsa** file will be saved. Click **Next** and then click **Finish.**

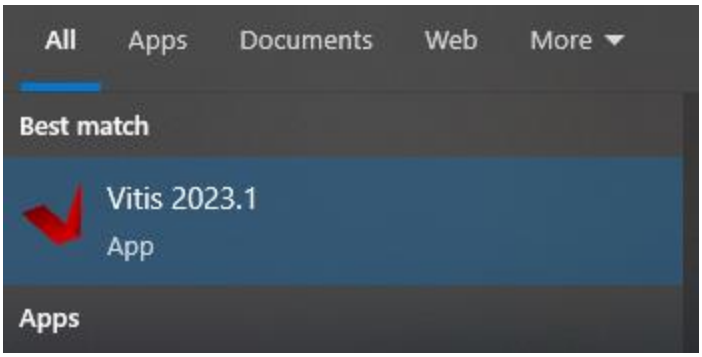


48. Verify that the **.xsa** file was successfully generated in the correct path.

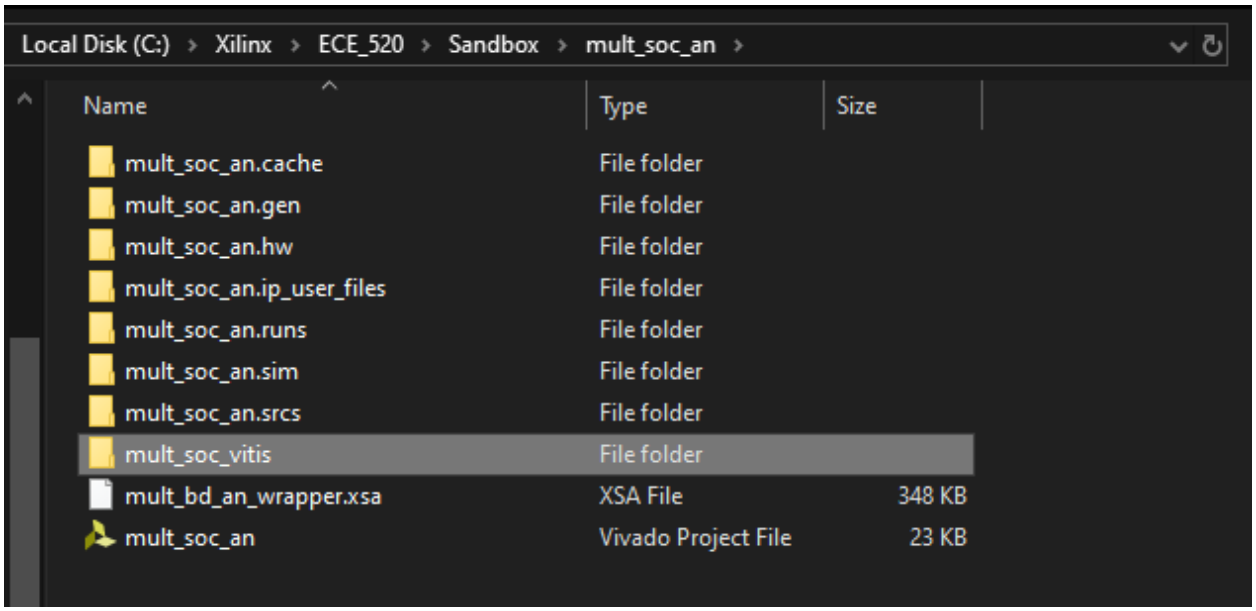


49. Search for **Vitis 2023.1** and launch the application. If you are using an older version of Vivado, then the application name might be **Vitis IDE**.

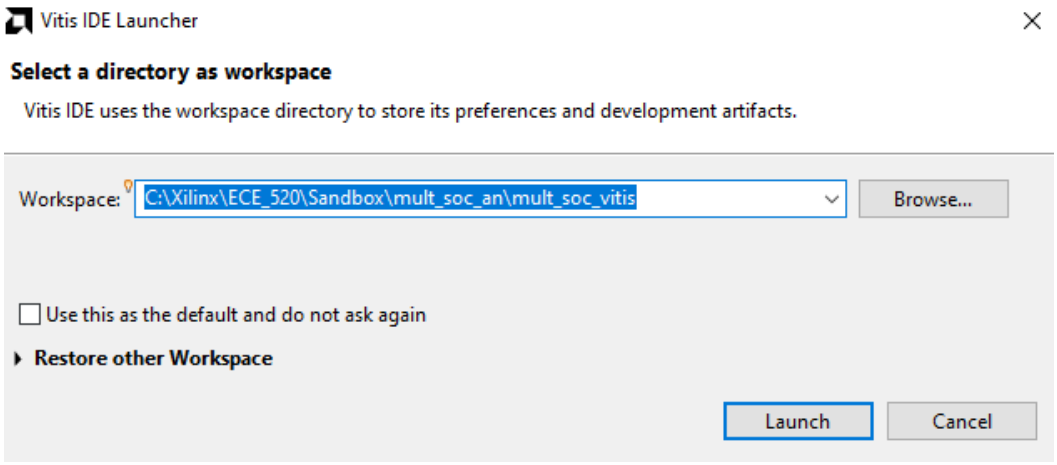
    Note: You are more than welcome to use the newest version of Vitis IDE instead of this version.



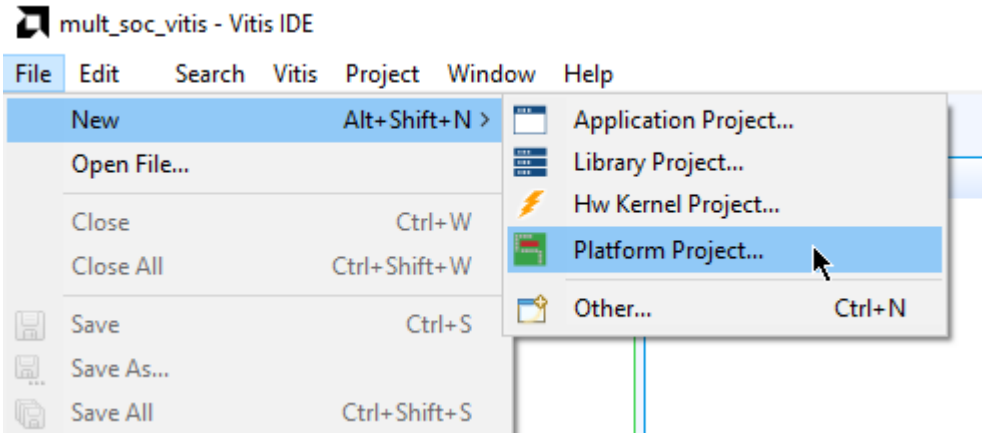
50. Create a folder for your Vitis workspace. In this example, a new folder named **mult_soc_vitis** inside the **mult_soc_<your_initials>** folder that was created.



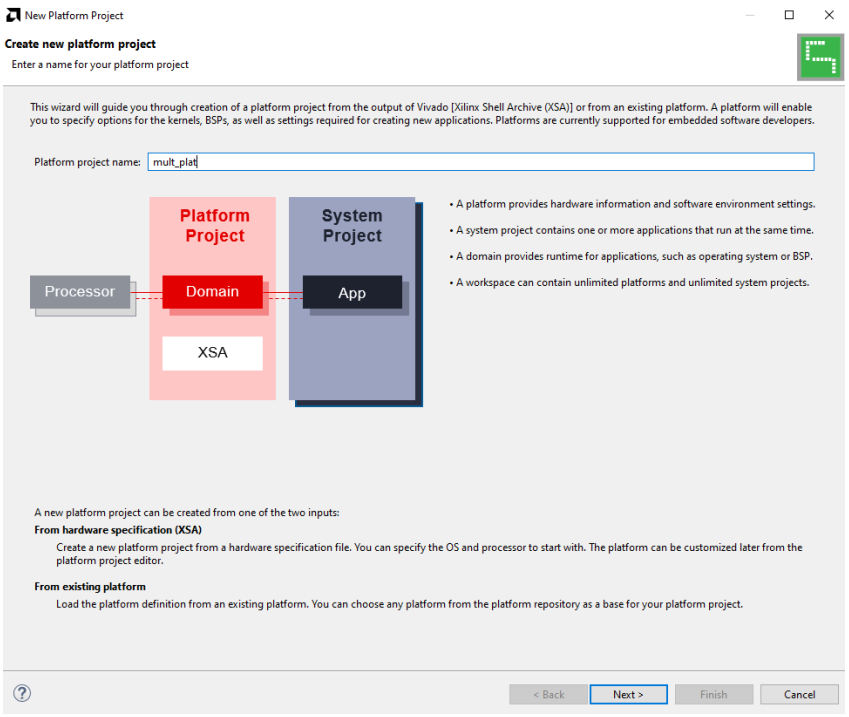
51. The Vitis IDE Launcher should appear. Select the Vitis folder that you've created and click **Launch**. An example is shown below.



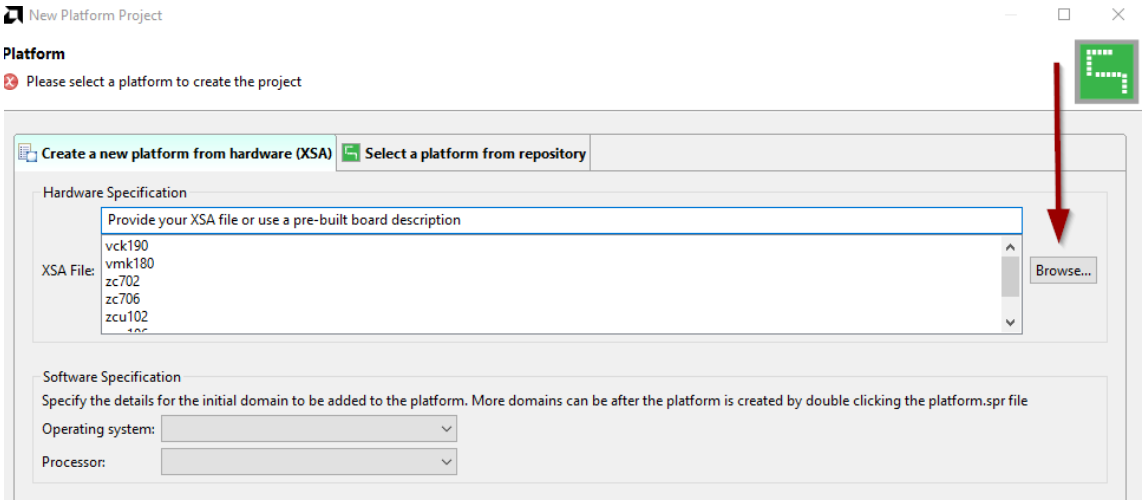
52. You will now create a platform project using the generated **.xsa** file. In Vitis IDE, select **File > New > Platform Project.**



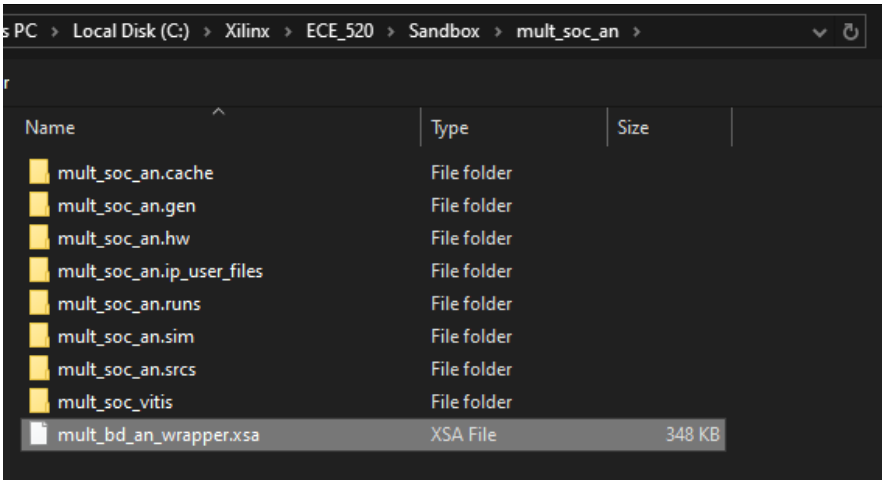
53. Enter a name for your platform project and click **Next**. In this example, the name of the platform project is **mult_plat**. Note that build errors may occur if the path names are too long.



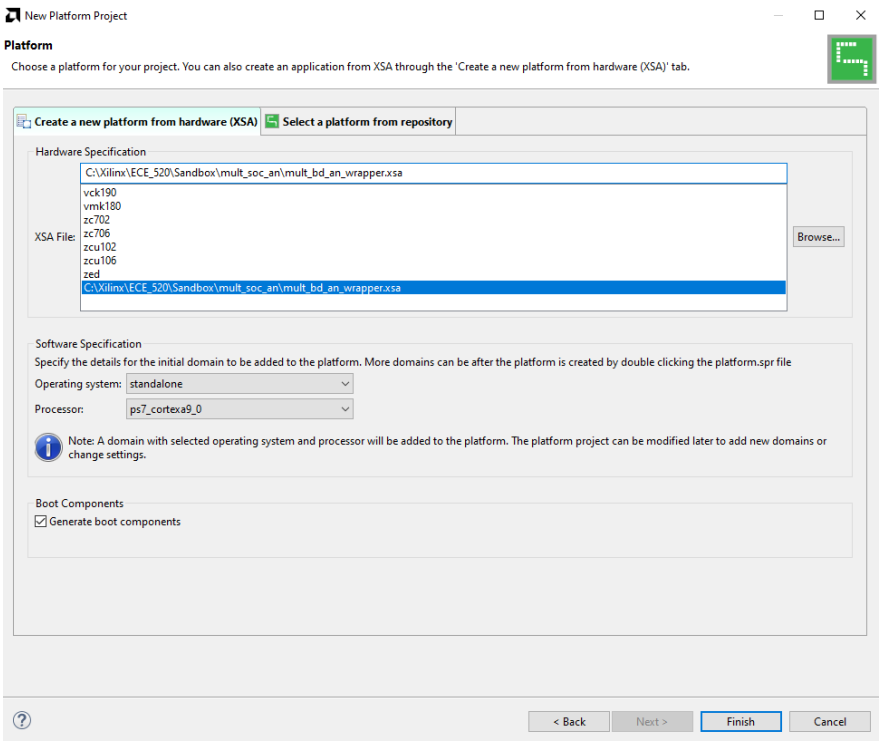
54. Select the **Create a new platform from hardware (XSA)** option and click on the **Browse** button.



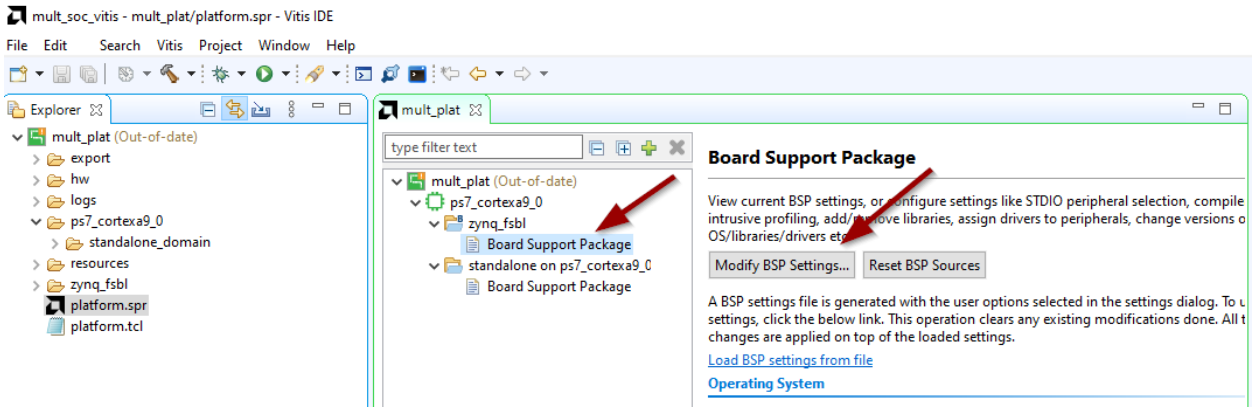
55. Choose the XSA file generated by Vivado and click **Open**.



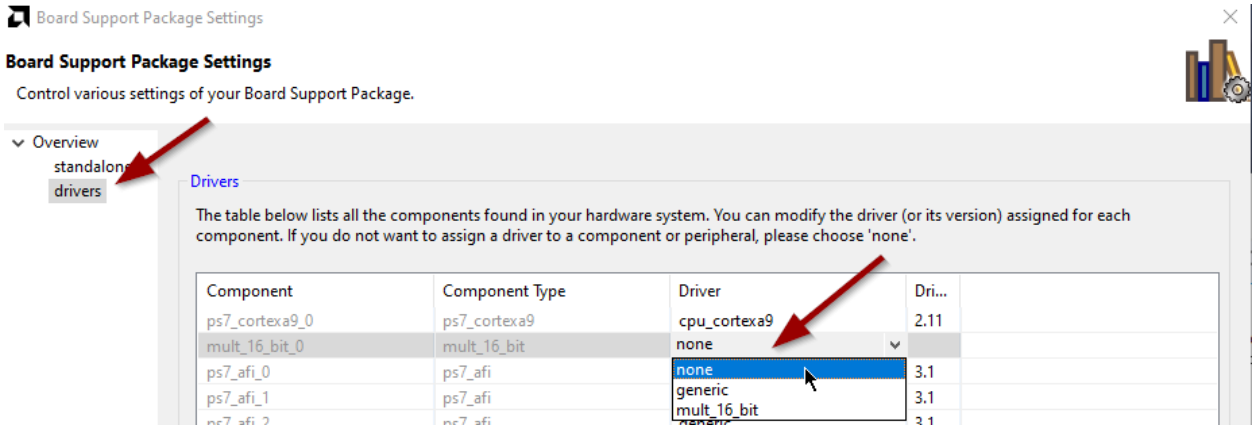
56. You should see a similar output as shown in the image below. Click **Finish** after the XSA file has been loaded.



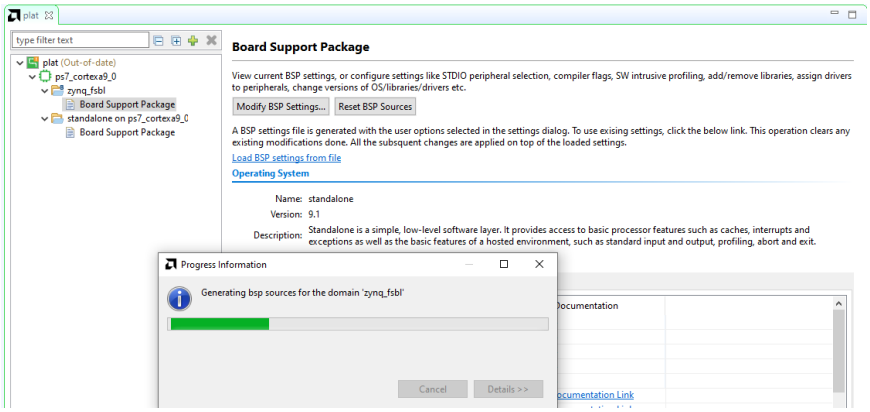
57. Under **zynq_fsbl**, select **Board Support Package** and click on the **Modify BSP Settings** button.



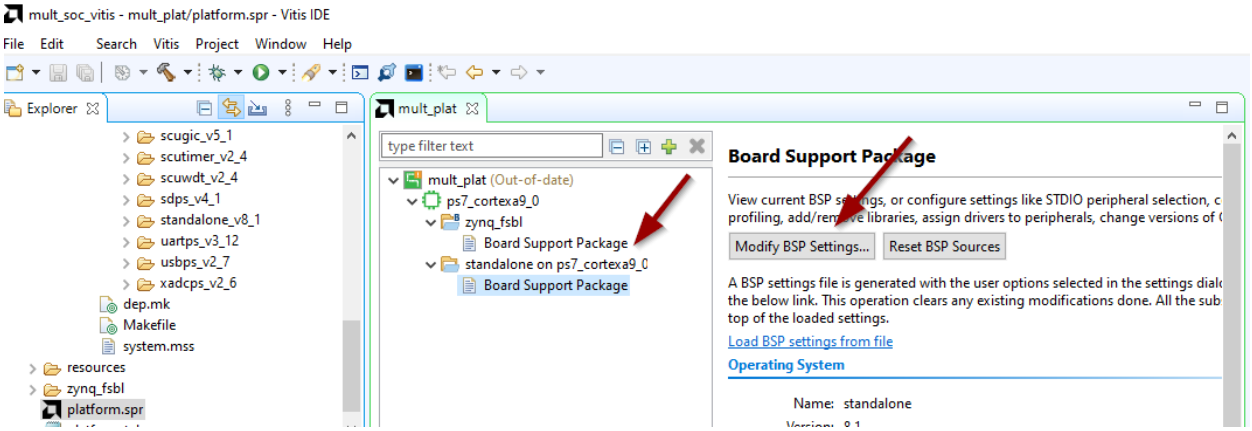
58. Click on **drivers** and look for the **mult_16_bit_0** component, select **none** for the driver. Hit OK.



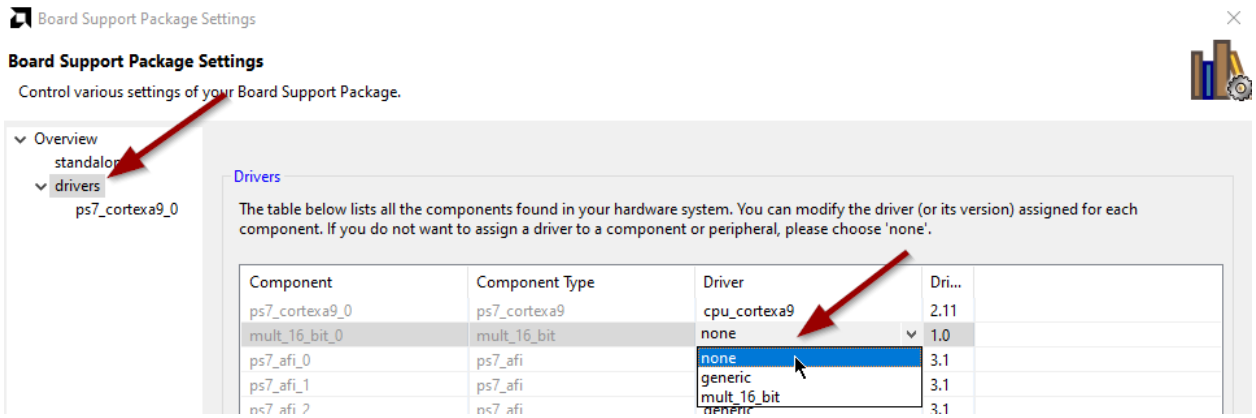
59. It will start regenerating the board support package files.



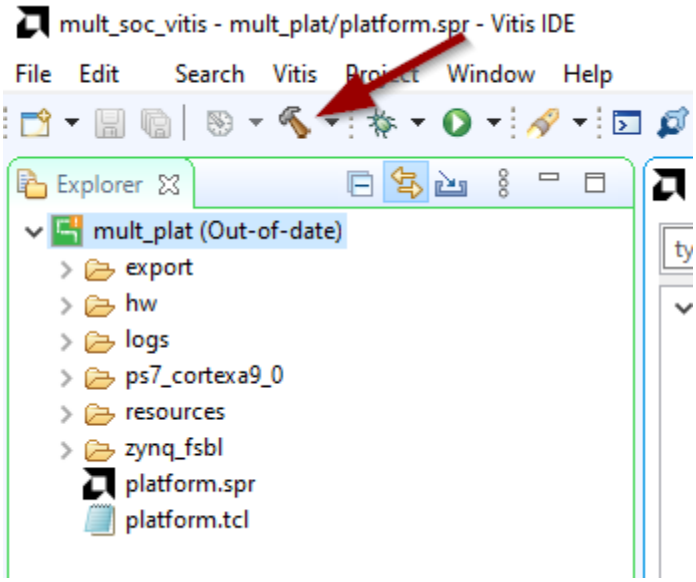
60. Next, select **Board Support Package** under **standalone on ps7_cortexa9_0** and click on the **Modify BSP Settings** button.



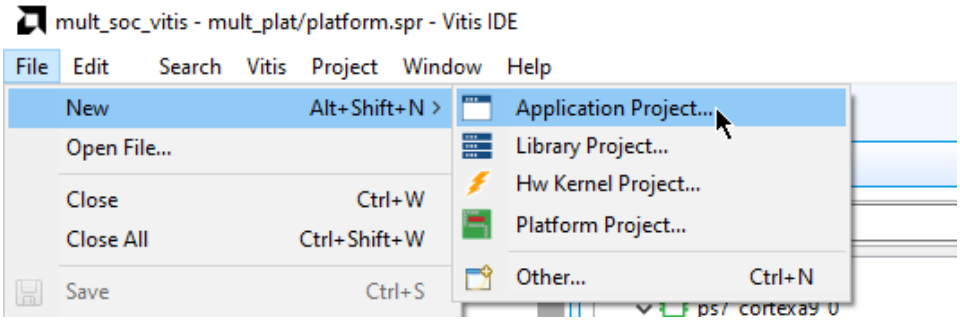
61. Similar to the previous steps, click on **drivers** and look for the **mult_16_bit_0** component, select **none** for the driver. Hit OK. It will regenerate the BSP sources again.



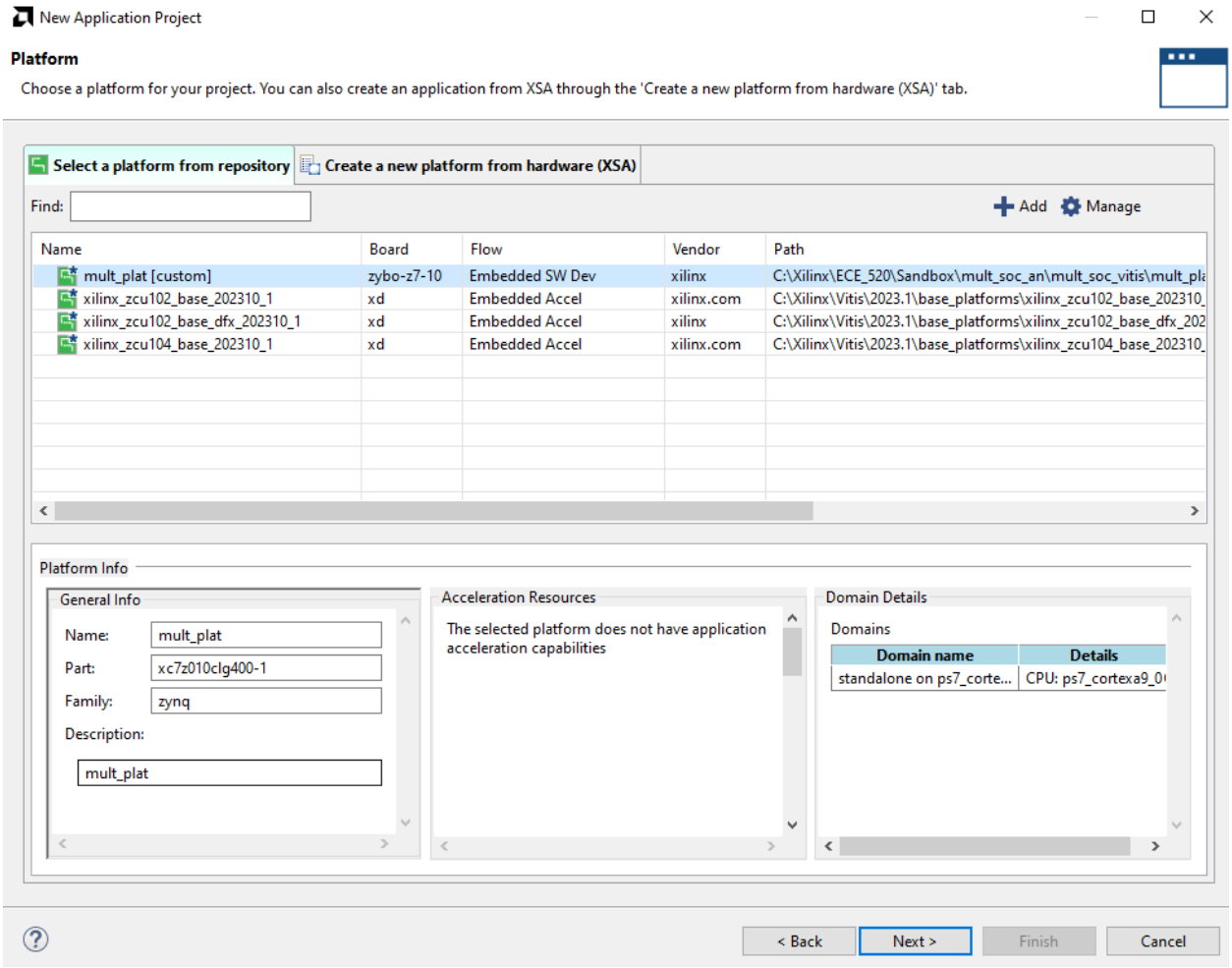
62. Build the platform project by clicking on the hammer button and verify that there are no errors in the build process.



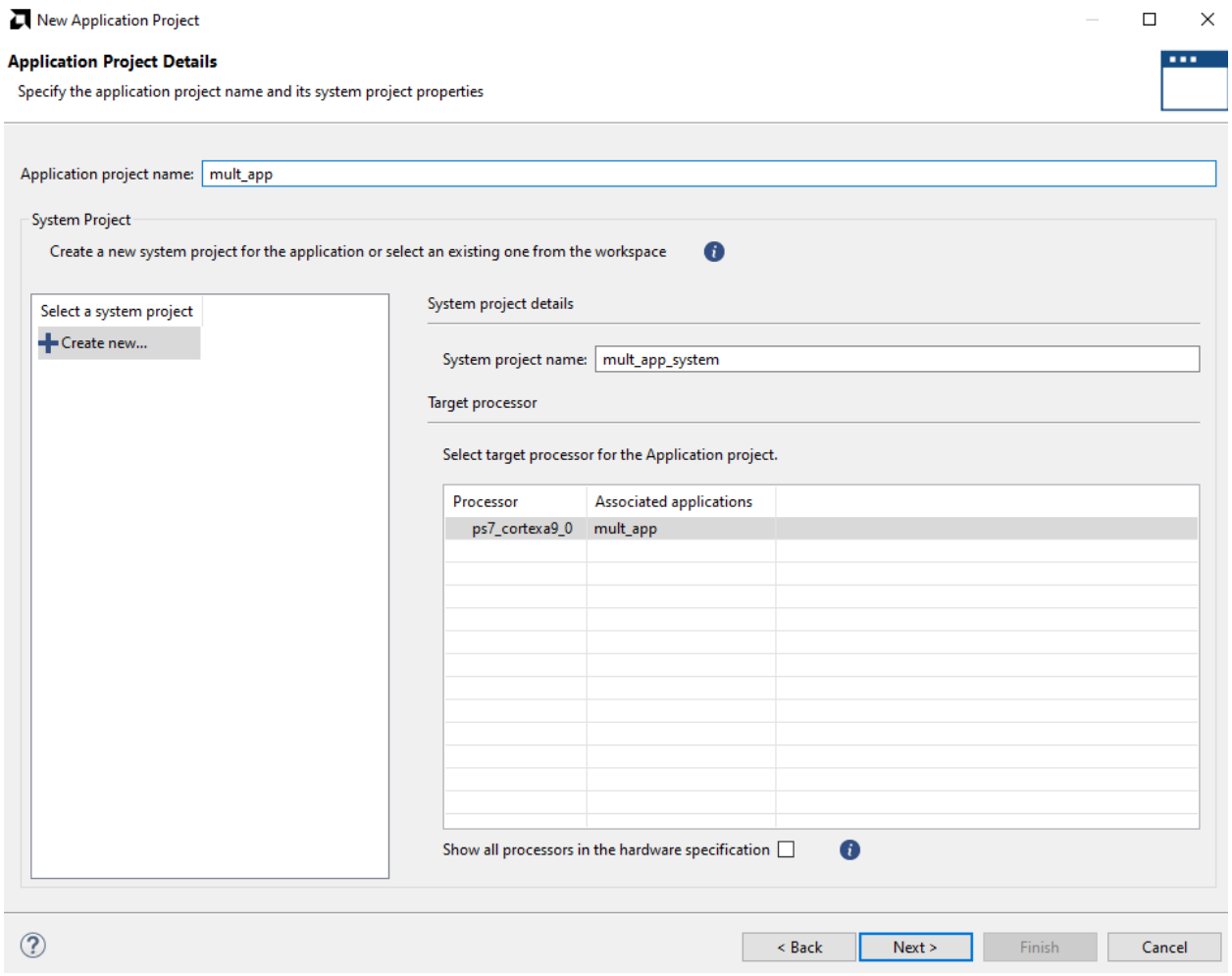
63. Next, you will create an application project. Click on **File > New > Application Project**.



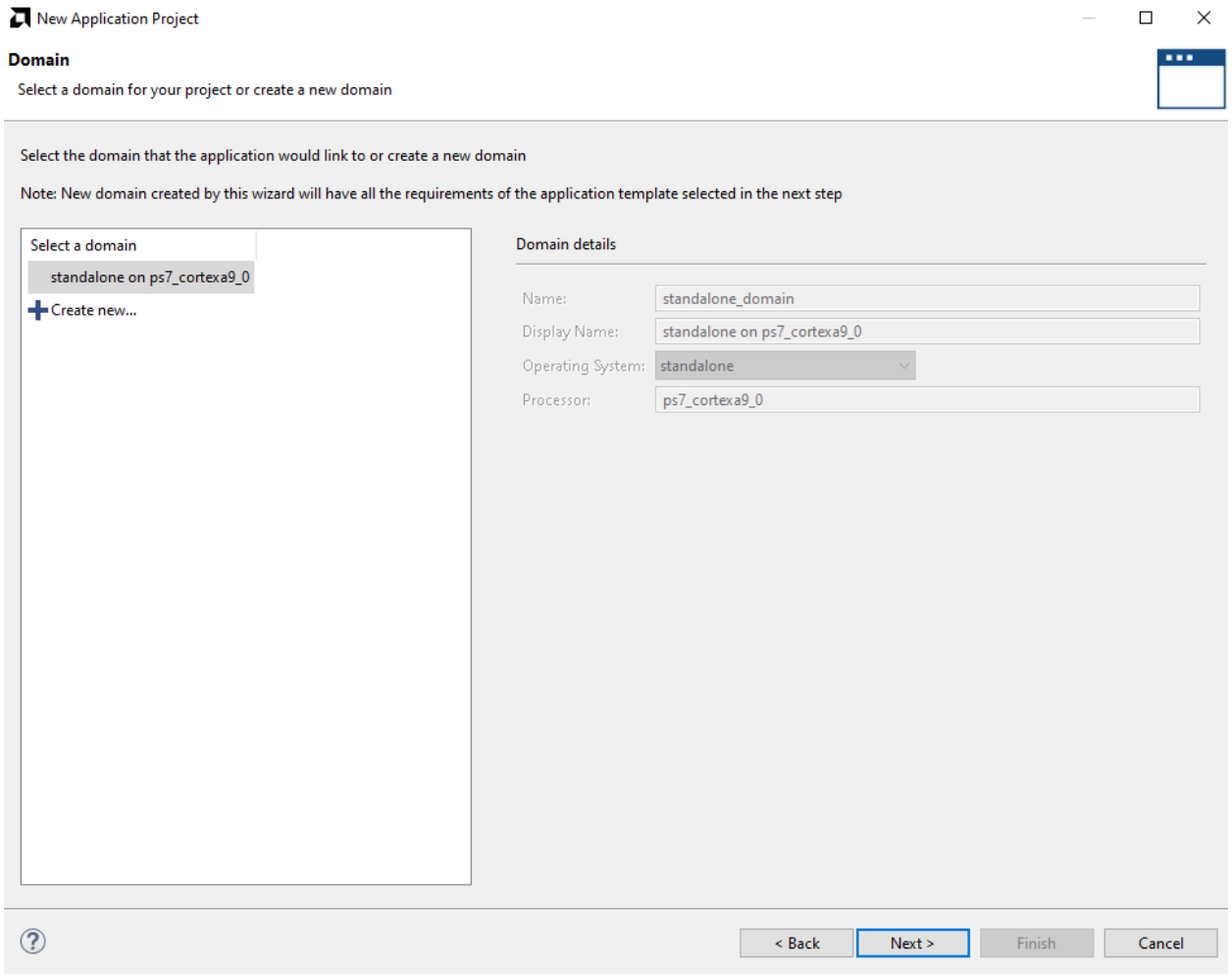
64. Select the **mult_plat [custom]** platform project and click **Next**.



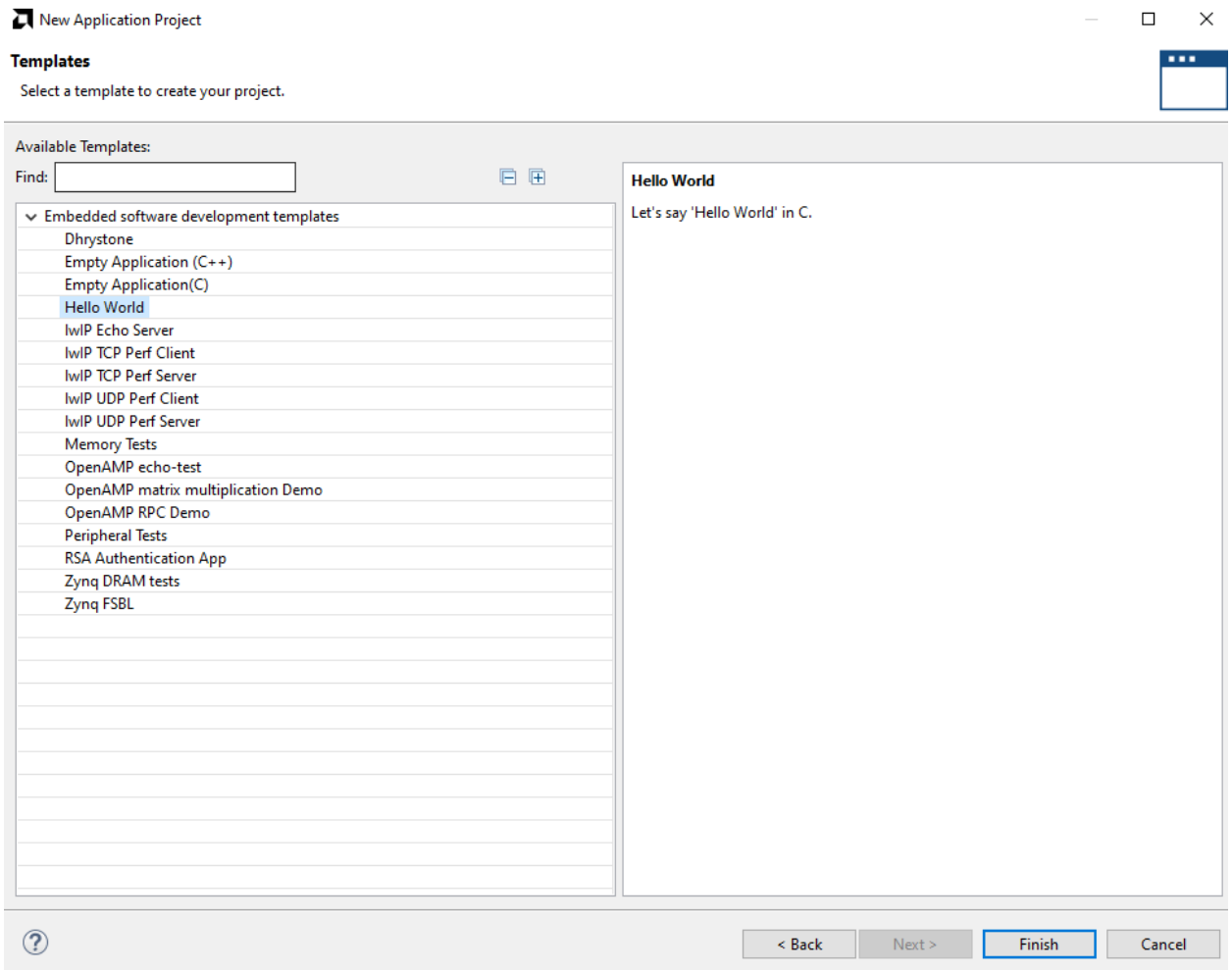
65. Enter a name for your application project. In this example, the name of the application project is **mult_app**. Click **Next**.



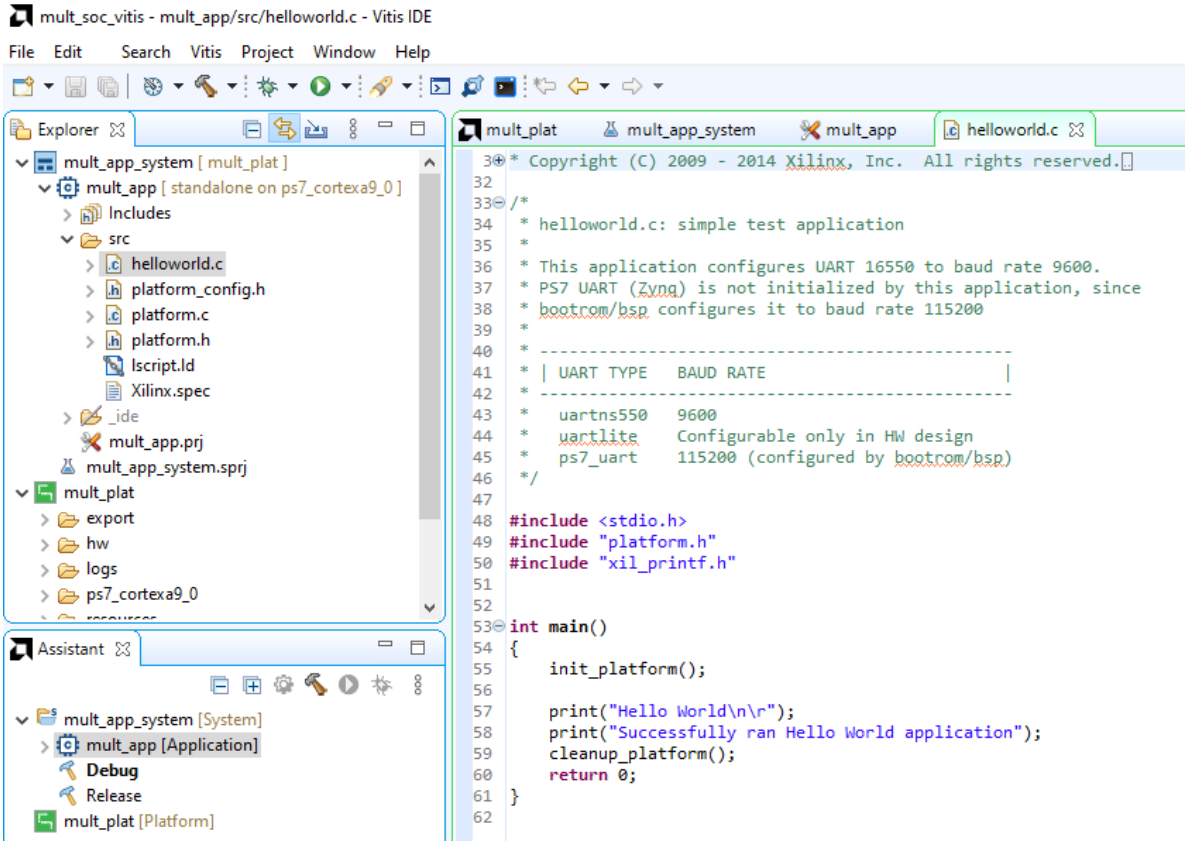
66. Select the default option for the domain and click **Next**.



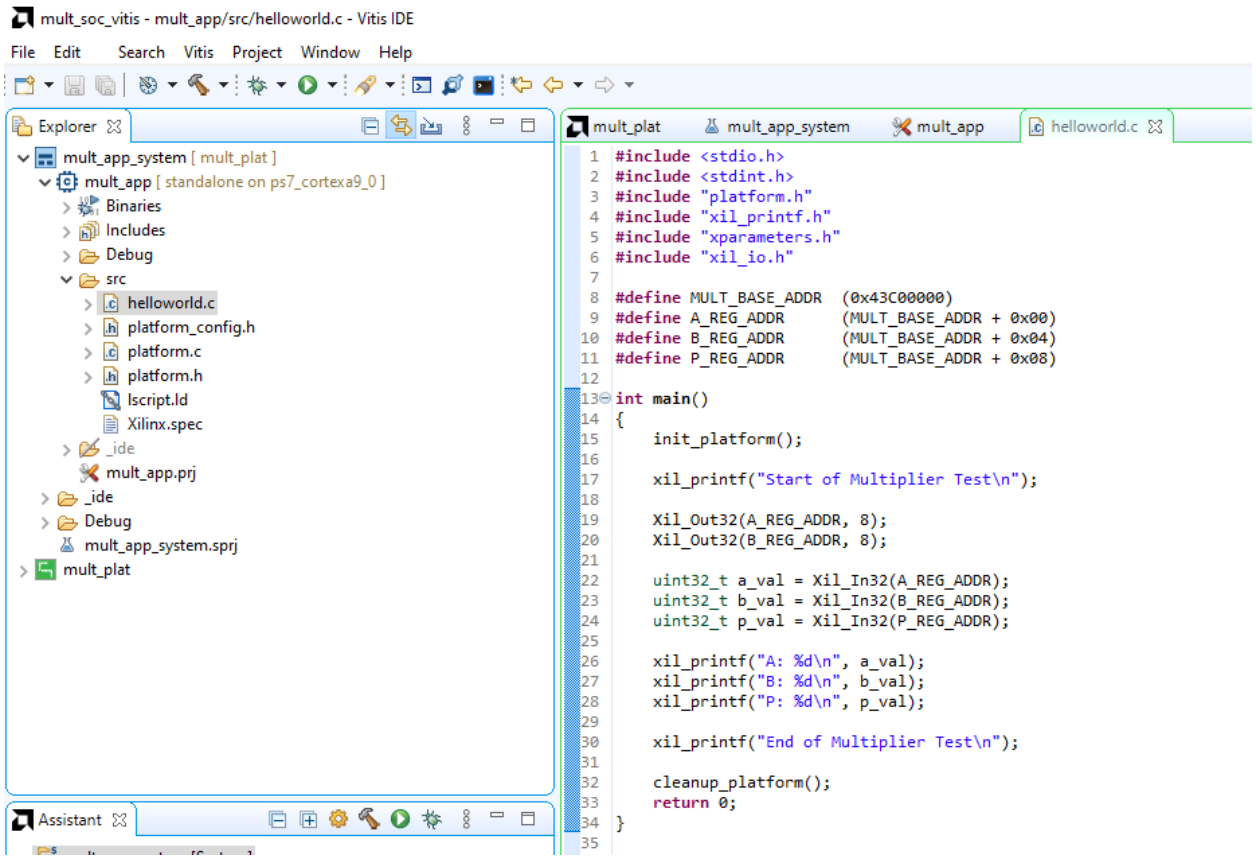
67. Select the **Hello World** template for the list of templates available and click **Finish**.



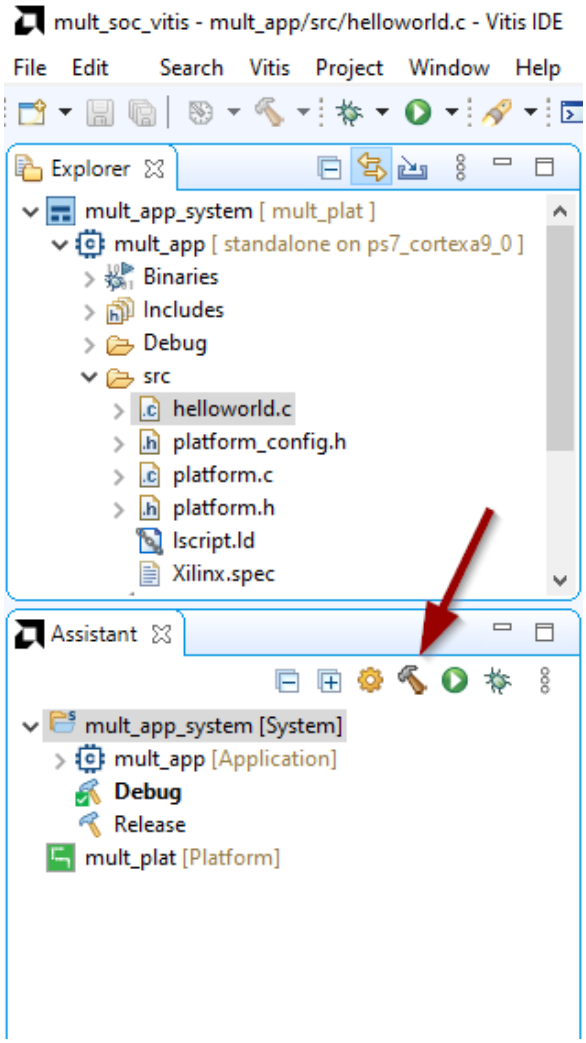
68. In the Explorer pane, you should now see **mult_app_system**. Expand the project and double-click on the **helloworld.c** source file to open it in the text editor. This file consists of a simple Hello World program in C.



69. To verify your 16-bit multiplier AXI IP, you will write a simple program that writes inputs to the A and B registers and prints the result using the Product register. Update the **helloworld.c** source file with the example C program shown in the image below.

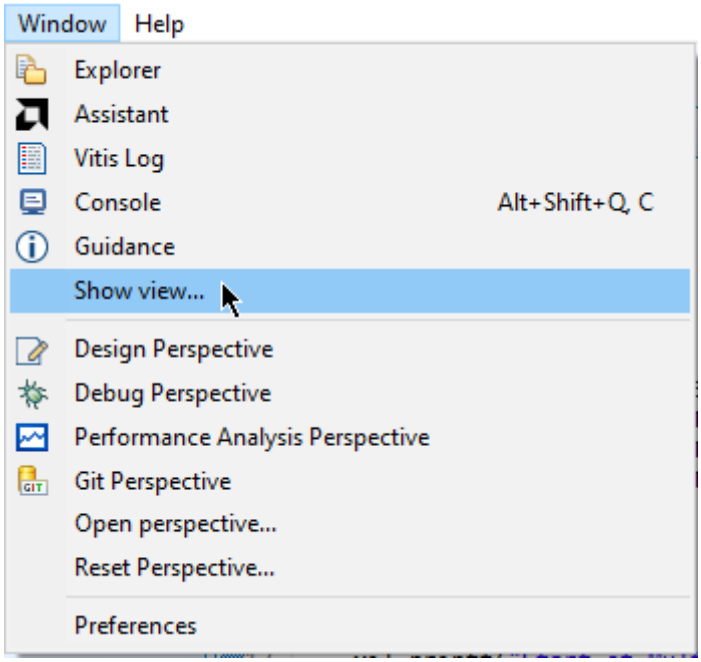


70. Build the project and all of its dependencies by clicking the hammer button as shown in the following image. Make sure that [System] is selected—this will cause each application in the system, as well as the platform, to be built.

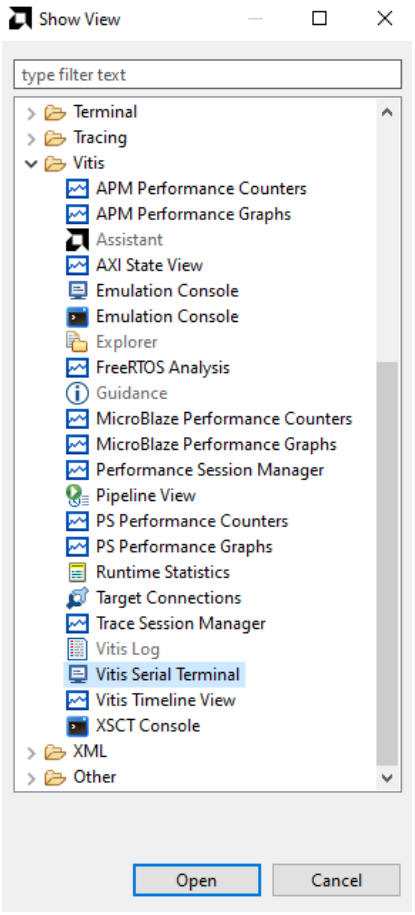


71. To see the **xil_printf** messages being printed out, you will need to open a COM port on a serial terminal.

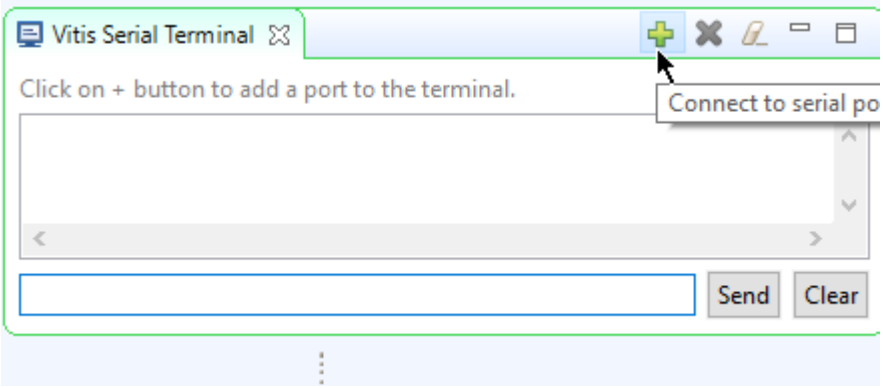
    To use the Vitis Serial Terminal, select the **Window > Show View** option.



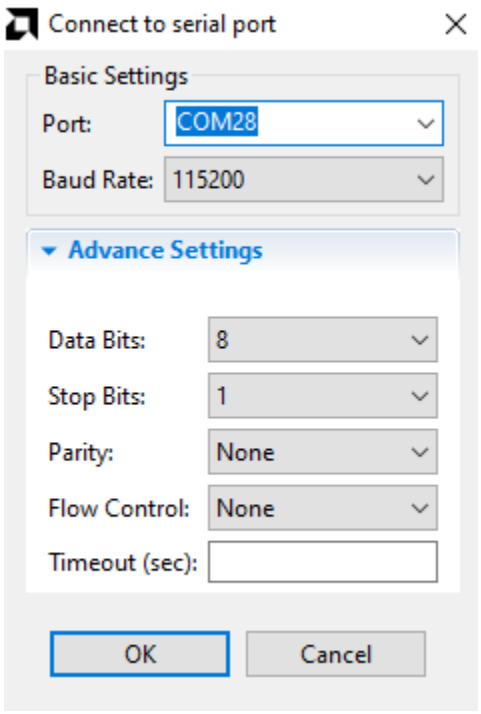
72. Under Vitis, select **Vitis Serial Terminal** and click **Open**.



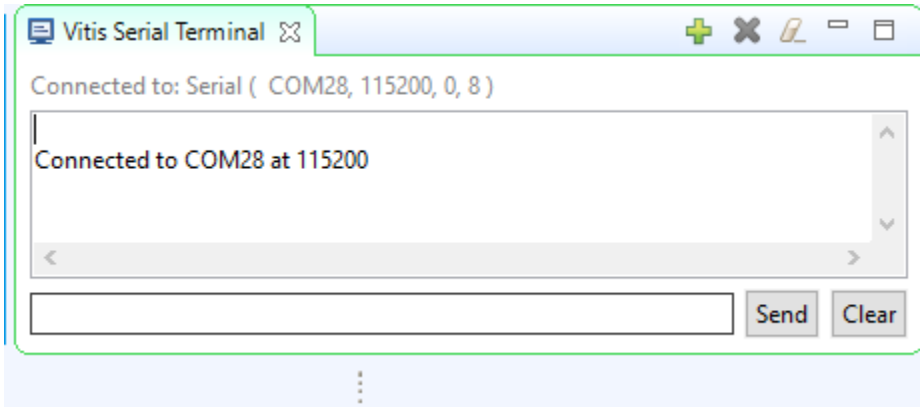
73. Click on the + button to configure the settings of the serial terminal.



74. Select the correct COM port and set the baud rate to 115200. Verify that the same settings apply in Device Manager. Hit OK.

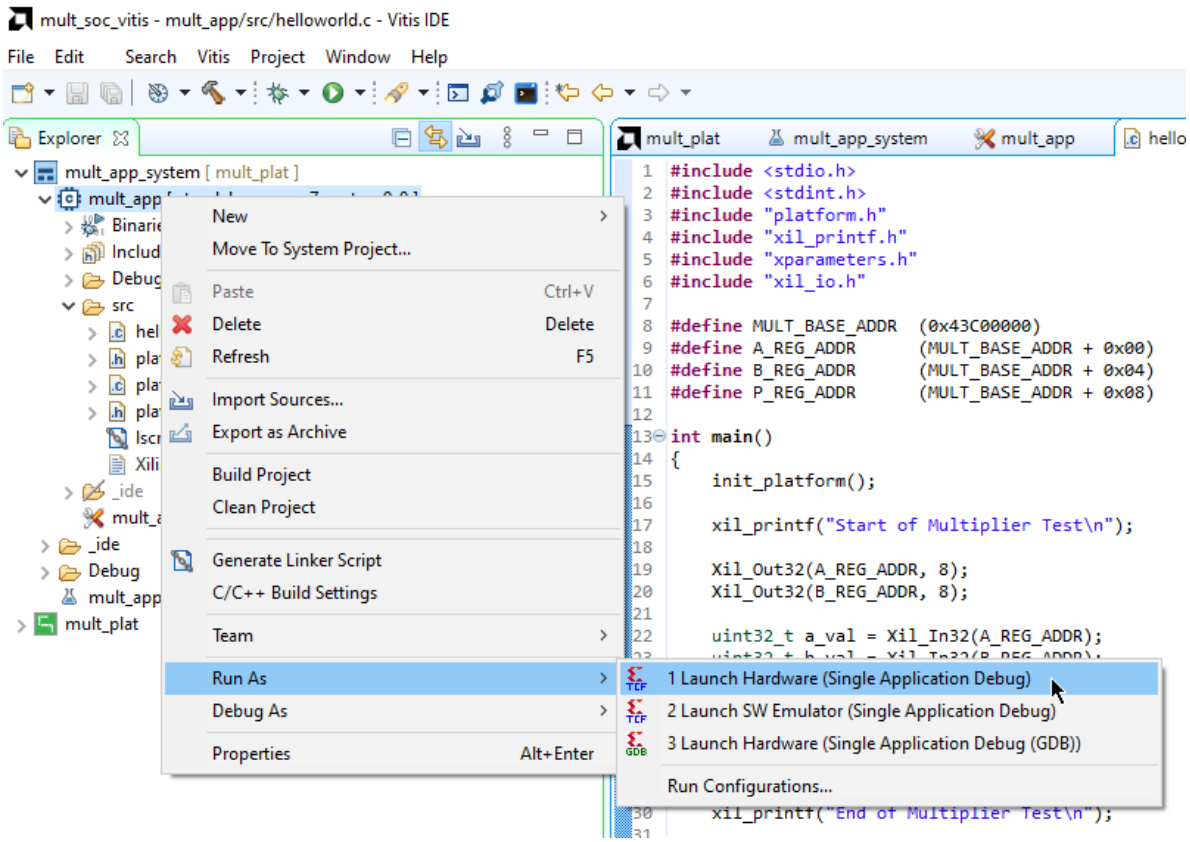


75. You should see the following verification message in the serial terminal.

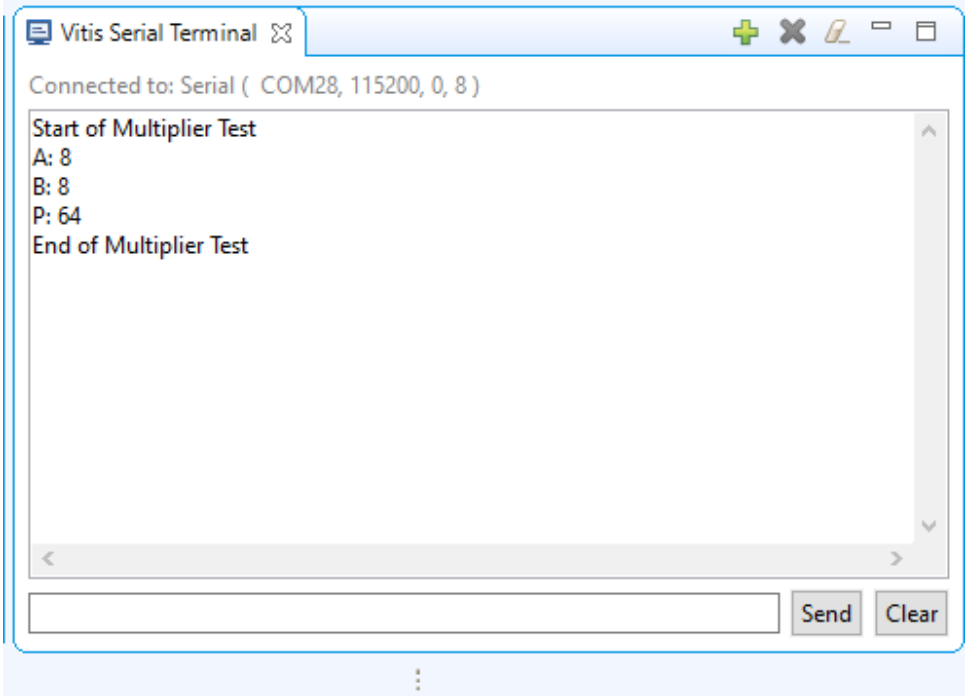


76. In the Explorer pane, right-click on the application and select the **Run As > 1 Launch Hardware (Single Application Debug)** option.

    This will program the FPGA with the bitstream and load the ELF file created by Vitis into the system memory. Then, the application will start to run.



77. You should see the following output on the serial terminal. Verify that you've completed this procedure with the instructor.



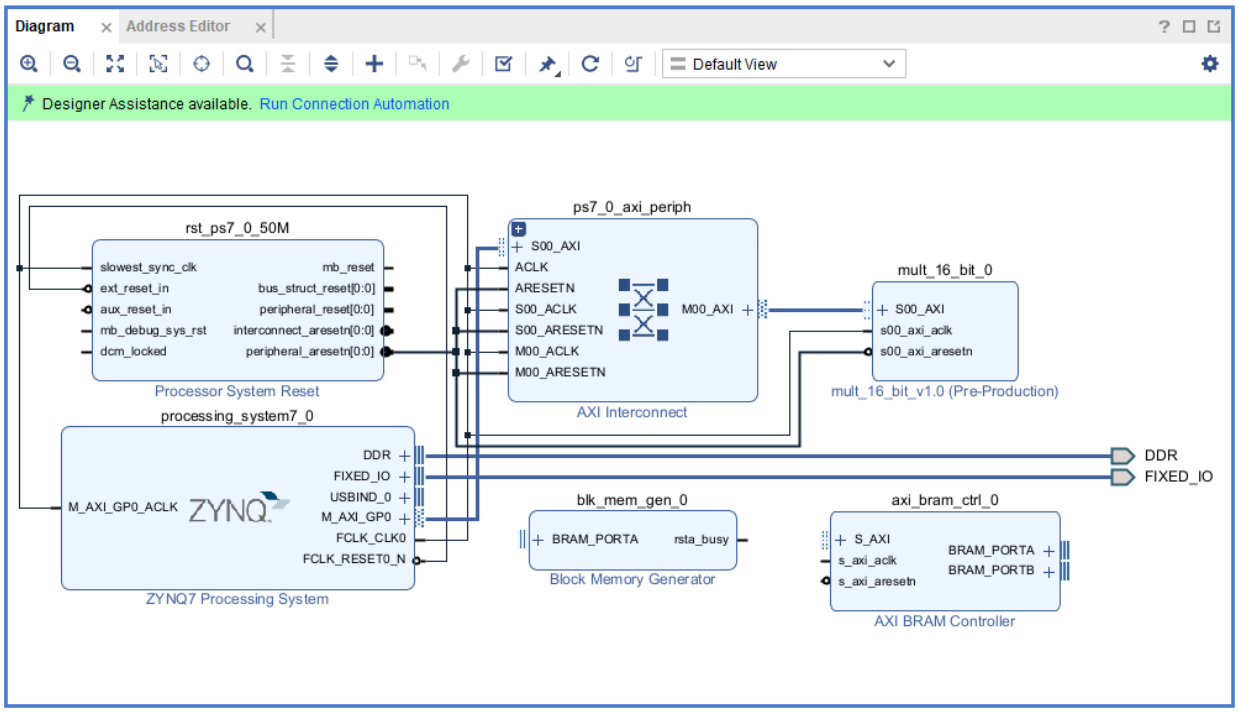
5. Tasks

You will extend your existing custom multiplier design by adding an AXI BRAM Controller and a Block Memory Generator (BRAM) to your Vivado block design. The results from the custom multiplier IP will be stored in the BRAM.

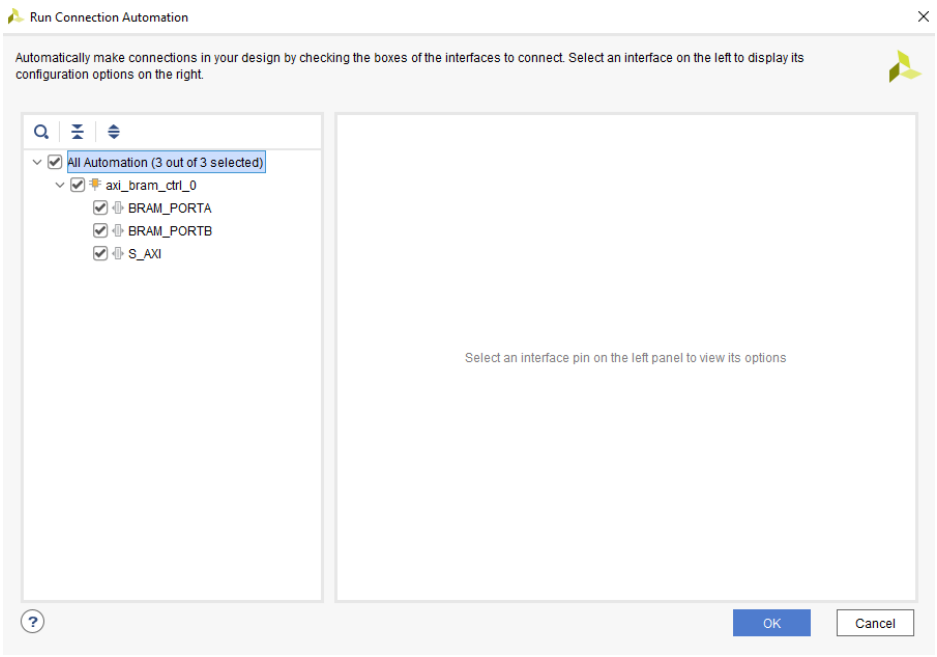
5.1. Hardware Tasks

Instantiate the following IP blocks to your block design. Click **Run Connection Automation**.

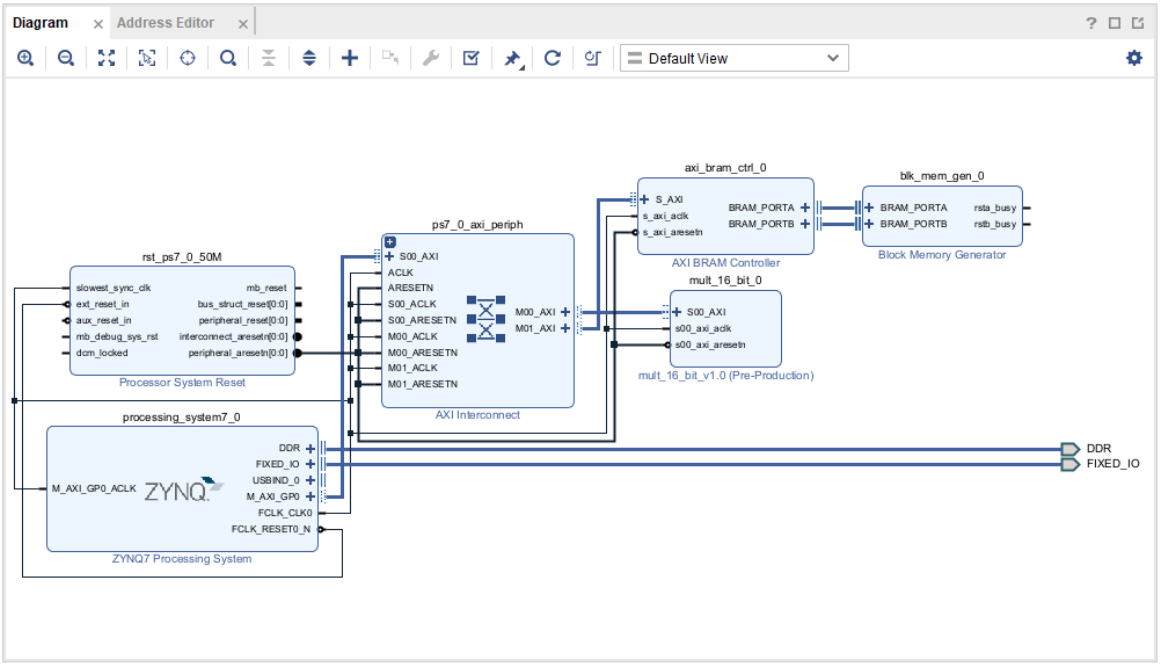
- AXI BRAM Controller
- Block Memory Generator (BRAM)



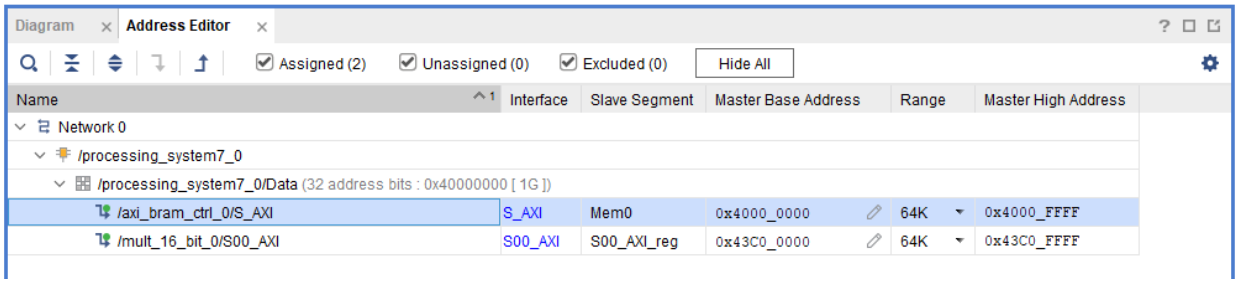
Select all of the options under **All Automation** and click OK.



An example block design diagram is shown below.



In Address Editor, modify the address range of the AXI BRAM IP to **64K.** Make note of the **Master Base Address** and the **Master High Address.**



In Address Editor, modify the address range of the AXI BRAM IP to **64K.** Make note of the **Master Base Address** and the **Master High Address.**

Validate the design, generate a new bitstream, and then export the hardware files. Using the new XSA file, create a new platform project in Vitis. Then, create a new application project. Note that when creating a new platform project, you will need to redo the steps for removing the drivers for your custom multiplier IP from the board support package.

5.2. Software Tasks

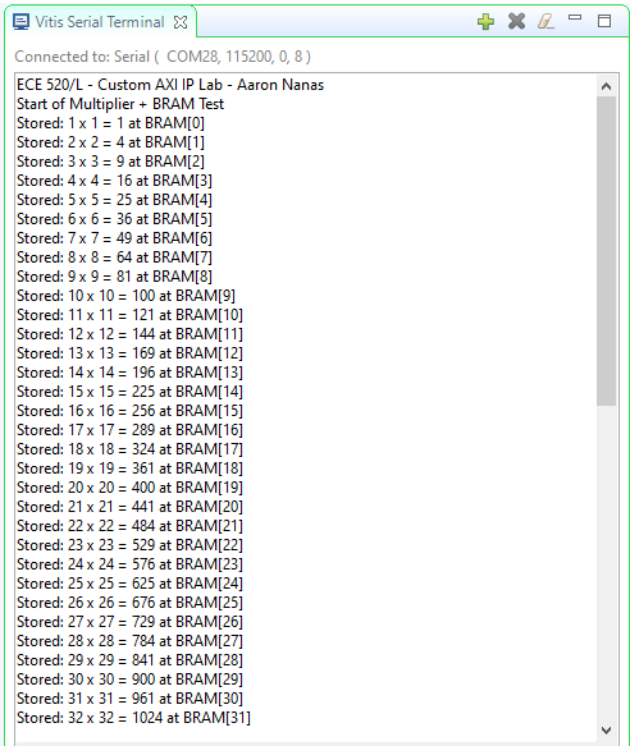
You will update your Vitis C program so that it performs 32 multiplication operations. Each product should be stored in the BRAM using word-addressing. Below is an example:

- Entry 0 (offset 0x00) stores product of 1 x 1
- Entry 1 (offset 0x04) stores product of 2 x 2
- …
- Entry 31 (offset 0x7C) stores product of 32 x 32

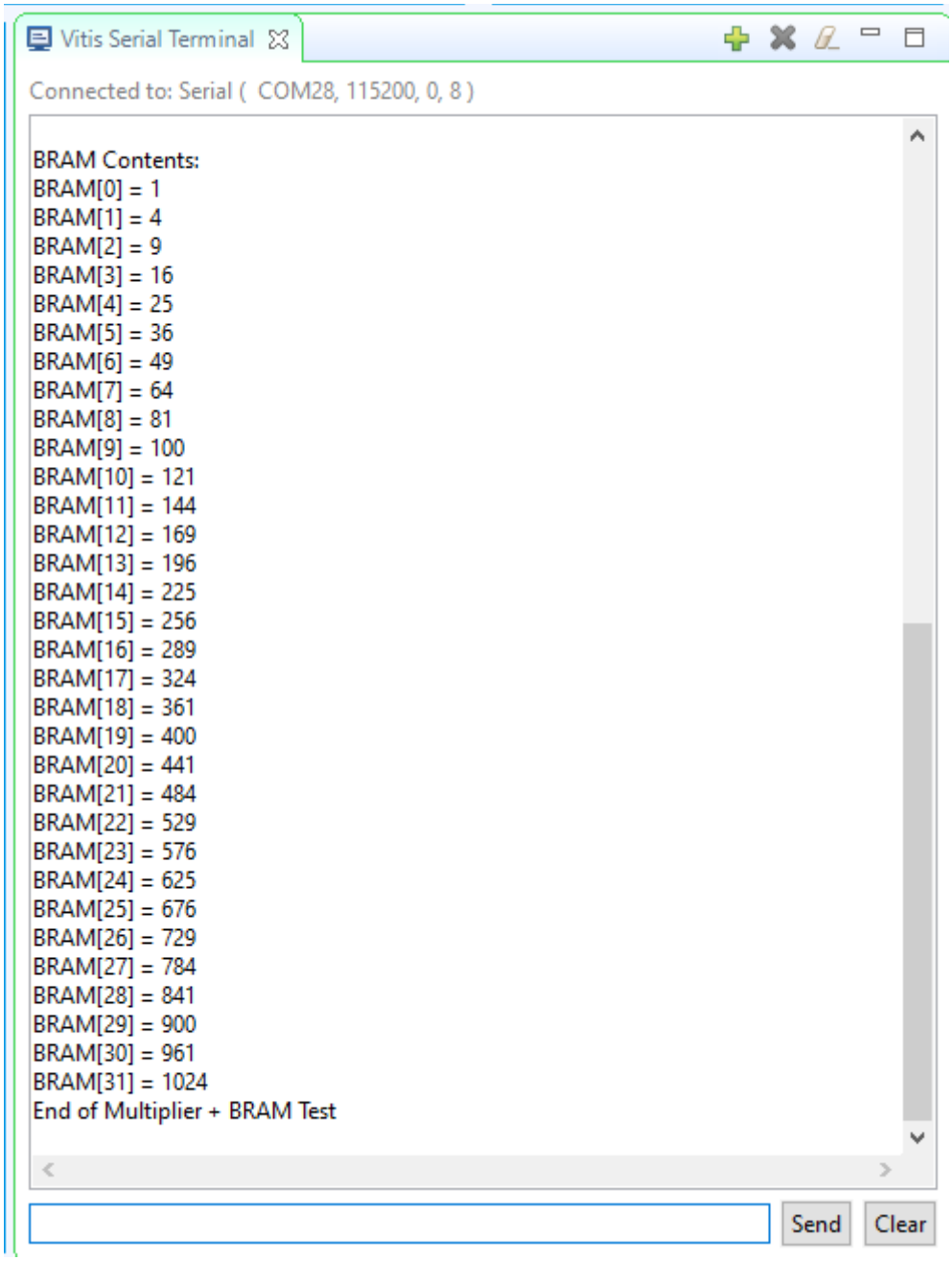
So the BRAM base address for entry *k* is:

- BRAM_BASE_ADDR + (k * 4)

Your program should print the contents of the BRAM after all multiplication operations have been completed. Loop through the BRAM entries 0 to 31 and then read each entry and print it on the serial terminal. Your name must be shown in the serial terminal output. Example serial terminal outputs are shown below. This output shows the numbers being multiplied and the products that are stored in BRAM.



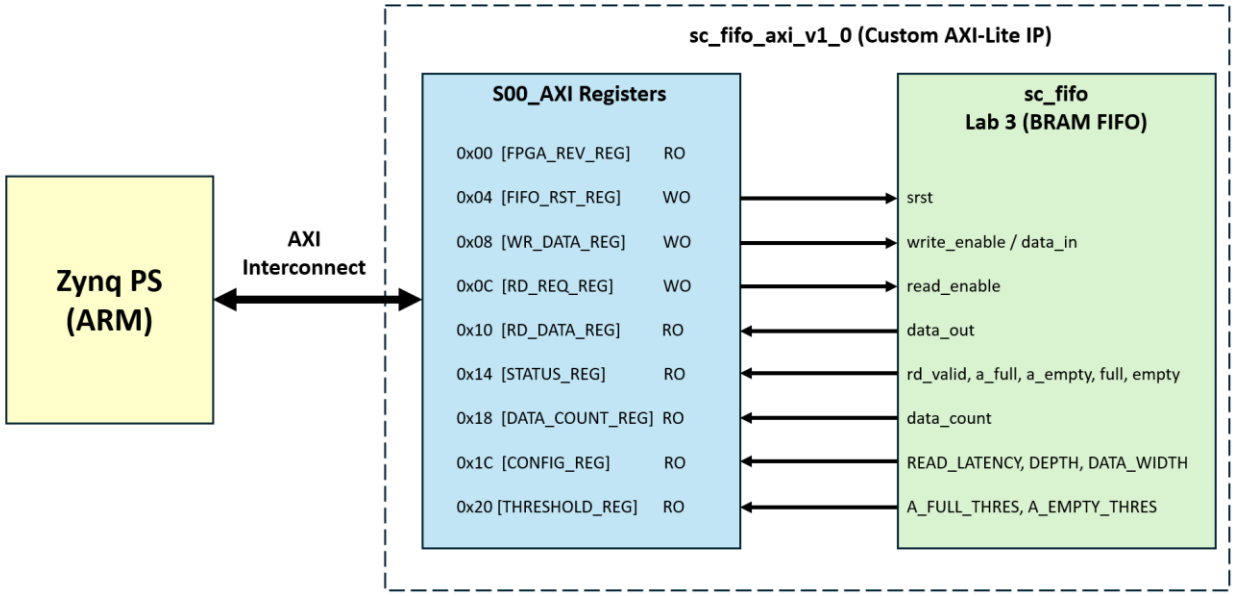
This output shows the contents of the BRAM.



6. Custom AXI-Lite FIFO IP

In the second part of this lab, you will package the BRAM FIFO that you designed in the previous lab as a second custom AXI4-Lite IP. Software will push data into the FIFO and pop data out of it through a set of memory-mapped registers. You will add the new IP to the same block design as your 16-bit multiplier IP. Then, you will verify it in simulation and test it on the Zybo Z7 board.

The block diagram of the custom AXI-Lite FIFO IP is shown below.



6.1. Design Specification

Your custom AXI-Lite FIFO IP must meet the following design requirements:

1. Name the IP sc_fifo_axi and use the same AXI4-Lite peripheral settings as the multiplier IP.

2. Instantiate your Lab 3 sc_fifo module inside the IP with its default parameters. The values reported in CONFIG_REG and THRESHOLD_REG must be derived from these parameters and not typed in as constants so that the IP will still report the correct values if any of the parameters are updated.

3. Connect the S_AXI_ACLK to the clock of the FIFO. Note that the AXI reset (S_AXI_ARESETN) is active low, while the Lab 3 FIFO reset signal (srst) is active high. The FIFO shall be reset when S_AXI_ARESETN is low or when software writes a 1 to the FIFO_RST_REG register.

4. Each write to the WR_DATA_REG register and each write of 1 to the RD_REQ_REG or FIFO_RST_REG registers shall assert write_enable, read_enable, or srst for exactly one clock cycle. A level held in a register would push or pop on every clock cycle, so the command signals must not be driven directly from slv_reg2, slv_reg3, or slv_reg1. You will need to include additional logic to perform single-clock pulses.

5. Since the FIFO output is registered and delayed by the READ_LATENCY parameter, popping a word will take two steps. Software must first write a 1 to the RD_REQ_REG register. When sc_fifo asserts valid, the IP will capture data_out into the RD_DATA_REG register and set the VALID bit of the STATUS_REG register.

6. Software can poll the RD_VALID bit and then read from the RD_DATA_REG register.

7. Software writes to read-only (RO) registers will have no effect. Write-only (WO) registers, reserved bits, and the unused offsets must always read as 0.

6.2. Register Map

The register map of the custom AXI-Lite FIFO IP is summarized below. All registers are 32 bits wide, and the offsets are relative to the base address assigned to the IP in Vivado's Address Editor tab. The access types are defined as follows:

- Read-Only (RO): The value comes from the IP. Software writes are ignored
- Write-Only (WO): A software write will trigger an action. Any reads will return as 0.

Offset	Register	Name	Access	Reset Value	Description
0x00	slv_reg0	FPGA_REV_REG	RO	0x5200_0004	FPGA revision value
0x04	slv_reg1	FIFO_RST_REG	WO	0x0000_0000	Software reset of the FIFO
0x08	slv_reg2	WR_DATA_REG	WO	0x0000_0000	Push one word into the FIFO
0x0C	slv_reg3	RD_REQ_REG	WO	0x0000_0000	Pop one word from the FIFO
0x10	slv_reg4	RD_DATA_REG	RO	0x0000_0000	Last word popped from the FIFO
0x14	slv_reg5	STATUS_REG	RO	0x0000_0005	FIFO flags and read-data valid
0x18	slv_reg6	DATA_COUNT_REG	RO	0x0000_0000	Number of words in the FIFO
0x1C	slv_reg7	CONFIG_REG	RO	0x0100_2010	FIFO parameters built into the IP
0x20	slv_reg8	THRESHOLD_REG	RO	0x001F_0001	Almost-full / almost-empty thresholds
0x24 – 0x3C	slv_reg9 – 15	RESERVED	–	0x0000_0000	Unused; reads as 0

6.2.1. FPGA Revision Register [FPGA_REV_REG]

Offset: 0x00 (slv_reg0)

Field	Bits	Access	Reset	Description
FPGA_REV_VALUE	[31:0]	RO	0x5200_0004	FPGA revision value. This is a hard-coded constant in the IP.

6.2.2. FIFO Reset Register [FIFO_RST_REG]

Offset: 0x04 (slv_reg1)

Field	Bits	Access	Reset	Description
RESERVED	[31:1]	–	0	Reserved. Reads as 0.
FIFO_RST	[0]	WO	0	Writing a 1 resets the FIFO. The srst signal is asserted for exactly one clock. This bit self-clears and always reads as 0. Writing 0 has no effect.

6.2.3. FIFO Write Data Register [WR_DATA_REG]

Offset: 0x08 (slv_reg2)

Field	Bits	Access	Reset	Description
RESERVED	[31:16]	–	0	Reserved. Reads as 0.
WR_DATA	[15:0]	WO	0x0000	Every write to this register pushes WR_DATA into the FIFO and write_enable is pulsed for exactly one clock. If the FIFO is full, the write is ignored and the FIFO state does not change. The field width follows the FIFO WIDTH parameter ([WIDTH-1:0]). Returns 0 when read.

6.2.4. FIFO Read Request Register [RD_REQ_REG]

Offset: 0x0C (slv_reg3)

Field	Bits	Access	Reset	Description
RESERVED	[31:1]	–	0	Reserved. Reads as 0.
RD_REQ	[0]	WO	0	Writing a 1 issues one read request, and read_enable is pulsed for exactly one clock cycle and clears the VALID bit of the STATUS_REG register. If the FIFO is empty, the request is ignored and RD_VALID remains 0. This bit self-clears and always reads as 0.

6.2.5. FIFO Read Data Register [RD_DATA_REG]

Offset: 0x10 (slv_reg4)

Field	Bits	Access	Reset	Description
RESERVED	[31:16]	–	0	Reserved. Reads as 0.
RD_DATA	[15:0]	RO	0x0000	Last word read out of the FIFO. Captured from data_out on the clock where sc_fifo asserts valid. This is only meaningful when STATUS_REG.RD_VALID = 1. Reading this register does not pop the FIFO.

6.2.6. FIFO Status Register [STATUS_REG]

Offset: 0x14 (slv_reg5)

Field	Bits	Access	Reset	Description
RESERVED	[31:5]	–	0	Reserved. Reads as 0.
RD_VALID	[4]	RO	0	This bit will be set when RD_DATA_REG has been loaded with a new word. Cleared by the next write of 1 to RD_REQ_REG, FIFO_RST, or AXI reset.
ALMOST_FULL	[3]	RO	0	This will reflect the status of the FIFO almost_full flag.
ALMOST_EMPTY	[2]	RO	1	This will reflect the status of the FIFO almost_empty flag.
FULL	[1]	RO	0	This will reflect the status of the FIFO full flag.
EMPTY	[0]	RO	1	This will reflect the status of the FIFO empty flag.

6.2.7. FIFO Data Count Register [DATA_COUNT_REG]

Offset: 0x18 (slv_reg6)

Field	Bits	Access	Reset	Description
RESERVED	[31:6]	–	0	Reserved. Reads as 0.
DATA_COUNT	[5:0]	RO	0	Number of words currently stored in the FIFO (0 to DEPTH). Field width is $clog2(DEPTH + 1) bits, which is 6 bits for DEPTH = 32.

6.2.8. FIFO Configuration Register [CONFIG_REG]

Offset: 0x1C (slv_reg7)

Field	Bits	Access	Reset	Description
RESERVED	[31:28]	–	0	Reserved. Reads as 0.
READ_LATENCY	[27:24]	RO	0x1	Value of the FIFO READ_LATENCY parameter.
DEPTH	[23:8]	RO	0x0020	Value of the FIFO DEPTH parameter.
DATA_WIDTH	[7:0]	RO	0x10	Value of the FIFO WIDTH parameter.

6.2.9. FIFO Threshold Register [THRESHOLD_REG]

Offset: 0x20 (slv_reg8)

Field	Bits	Access	Reset	Description
ALMOST_FULL_THRESHOLD	[31:16]	RO	0x001F	Value of the ALMOST_FULL_THRESHOLD parameter.
ALMOST_EMPTY_THRESHOLD	[15:0]	RO	0x0001	Value of the ALMOST_EMPTY_THRESHOLD parameter.

7. References

1. Zybo Z7 Product Page
    - Link
2. Zybo Z7 Reference Manual
    - Link
3. Creating and Packaging Custom IP User Guide
    - Link